Agent Memory - Richmond Alake

Wednesday, August 20, 2025 11:55 PM

Follow : https://www.linkedin.com/in/richmondalake/ for remaining 70 days topic

Day 1/100 of Agent Memory

It turns out that many developers, practitioners, and founders are very interested in making AI Agents reliable, believable, and capable. And we all agree that memory is a core component in making this happen.

What's Agent Memory?
AI agent memory is the persistent cognitive architecture that allows agents to accumulate knowledge, maintain contextual awareness, and adapt their behavior based on historical interactions and learned experiences. Think of it as giving AI a working memory that persists beyond a single interaction.

Why does Agent Memory matter?
As I said earlier, memory makes an AI agent reliable, believable, and capable.
→ Without memory, every conversation with an AI agent starts from scratch.
→ Without memory, AI agents can't adapt to new information
→ Without memory, AI agents can't be personalized
→ Without memory, AGI won't happen

Documenting my deep dive into key definitions, memory management techniques, research findings, implementation strategies, and real-world applications - sharing both insights and mistakes along the way.

Day 2/100 of Agent Memory

From Context to Memory Engineering
Context Engineering is the buzzword of the week in AI circles, and for good reason. We're finally getting closer to the real work that needs to be done, but we need to go one step further to build truly reliable, believable, and capable AI agents.

What is Context Engineering?
Philipp Schmid describes it as "the discipline of designing and building dynamic systems that provides the right information and tools, in the right format, at the right time, to give a LLM everything it needs to accomplish a task."

Tobias Lütke, CEO of Shopify, captures the essence even more succinctly: "the art of providing all the context for the task to be plausibly solvable by the LLM."

And Andrej Karpathy blessed it.
So you know it's going in the Oxford dictionary along with Vibe Coding.

The Limitation of Context Engineering
The fundamental limitation: context engineering optimizes for individual interactions, but it doesn't enable systems that learn and evolve over time. Your AI agent handles a complex customer inquiry perfectly, but forgets everything about that interaction.

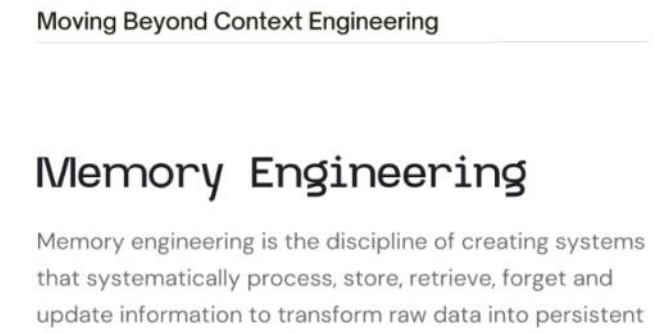
Enter Memory Engineering
Memory engineering addresses this by moving AI systems from stateless optimization to persistent intelligence.

Instead of just asking:
❌ What information does this system need right now?
Memory engineering asks:
✅ How can this system accumulate understanding that makes it more reliable and capable over time?

Here's what it looks like in practice↴
A context-engineered customer service agent retrieves relevant policy documents and previous tickets to answer current questions.
A memory-engineered system does all of that, plus remembers interaction patterns, learns which communication approaches work best with specific customers, and continuously refines its understanding of common problem resolution strategies.

Memory engineering is the discipline of creating systems that systematically process, store, retrieve, and update information to transform raw data into persistent memory components, then dynamically retrieve and assemble the most relevant memory components to populate an LLM's context window for each interaction or session.

To be clear, Memory engineering doesn't replace context engineering
It builds on it.



Moving Beyond Context Engineering

Memory Engineering

Memory engineering is the discipline of creating systems that systematically process, store, retrieve, forget and update information to transform raw data into persistent memory components, then dynamically retrieve and assemble the most relevant memory components to populate an LLM's context window for each interaction or session.

This engineering discipline enables the development of AI agents and systems that continuously evolve their understanding and capabilities through accumulated experience, making them more adaptive, effectively ensuring they become more valuable and effective with every interaction.

Moving Beyond Context Engineering

Memory engineering extends context engineering by adding a persistence layer that enables learning and adaptation over time.

While context engineering optimizes what information goes into the context window of an LLM for individual interactions—getting the right information, in the right format, at the right time—**memory engineering captures the outcomes of those interactions and transforms them into accumulated understanding that improves future performance.**

Think of context engineering as perfecting each conversation, while memory engineering enables the system to remember what worked, learn from mistakes, and build genuine expertise across conversations.

Context engineering asks "what information does this system need right now?" while memory engineering asks "how can this system learn from this interaction to become more capable in future interactions?"

Day 3/100 of Agent Memory

Let's get you explaining Agent Memory like a pro.

Before we dive deeper into Agent Memory
It helps to understand why it matters in the first place.

Whenever I'm asked about the importance of memory in AI

I explain it this way:
If the aim of artificial intelligence is to mimic human intelligence, and one of the core components of intelligence is the ability to retain, recall, and reuse information—which in humans is governed by memory—then you can make the argument that for AI to achieve its objective, it needs some form of augmented computational memory.

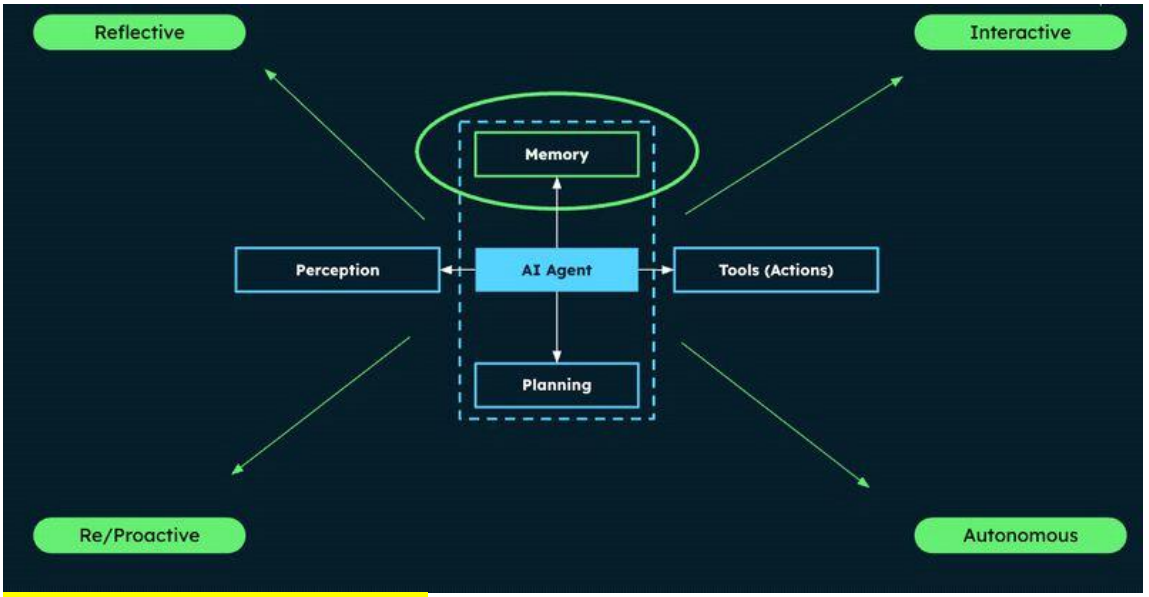
Today, the agreed-upon form factor of AI is predominantly AI Agents and agentic systems.

Why Memory is Non-Negotiable
The importance of memory becomes crystal clear when you understand that it's the core component that makes AI agents more Reliable, Believable, and Capable—I call this the RBCs of AI Agents.
1 Reliable through consistent access to relevant historical context and learned patterns
2 Believable through coherent behavior that reflects accumulated experiences
3 Capable through the ability to leverage past knowledge for complex reasoning

Memory Types Powering RBC Agents
Different memory types enable different aspects of RBC behavior:
1 Reliability is powered by Episodic Memory - storing specific interactions with timestamps and context, ensuring agents can consistently reference past events and maintain coherent decision-making patterns over time.
2 Believability emerges from Semantic Memory - accumulating general knowledge and relationships that enable agents to demonstrate understanding that feels natural and contextually appropriate, just like humans draw from learned concepts. Persona memory also plays a role here, more on this later.
3 Capability is enhanced by Procedural Memory - retaining learned skills, workflows, and problem-solving approaches that allow agents to execute complex tasks more effectively through accumulated experience.

A Learning Exploration in Real Time
Full transparency: I'm not an expert teaching unsettled science.
I'm an engineer actively exploring and uncovering these concepts alongside the broader AI community.

So far, I find that this journey is as much about asking the right questions as it is about finding answers.



Day 4/100 of Agent Memory

Let's talk about Application mode
Their importance and how they map to Memory Types.

I've been thinking a lot about how the application mode an agent is expected to run in fundamentally defines the memory architecture it needs.

What is Application Mode?
Application mode refers to the operational environment and interaction patterns an agent is designed for. It's not about the domain (legal vs. medical vs. coding) but about how the agent fundamentally operates.
The three core application modes I'm observing are:
1 Assistant - Task-oriented systems providing specialized help through interactive dialogue while maintaining user context
2 Workflow - Process-driven systems that orchestrate multi-step operations through structured execution paths and tool coordination
3 Deep Research - Multi-source investigation systems that gather, synthesize, and report findings over extended periods

Why Application Mode Matters
Identifying the right application mode when designing agent memory affects downstream decision in your AI stack:
✅ Embedding models - General purpose vs. domain-specialized
✅ Memory architecture - Which memory types your agent needs
✅ Agentic architecture - Single agent vs. multi-agent orchestration
✅ Retrieval methods - How your memory provider should be optimized

For example, Deep Research mode typically requires multi-agent architecture due to long-running execution and query parallelization. You spawn multiple agents for

different research aspects, consolidate findings in shared memory, and have an orchestrator synthesize the final report.

When exploring the topic of application mode in agent memory I made I mistake. I viewed Coding, as a mode, which is wrong, it's instead a subset of Assistant Mode. I realized this when I making some changes to MemoRizz and trying to consolidate patterns in how agents are initialized.

💡So, the insight here is coding assistants, legal assistants, and medical assistants are all domain-specific variants of Assistant mode—they share the same operational patterns and memory requirements, just with different specialized knowledge.

#100DaysOfAgentMemory

What Is Application Mode?

Application mode refers to the distinct operation pattern that defines how an AI agent interacts with users and processes information, characterized by specific behavioral expectations, interaction styles and corresponding memory architecture requirements.

Primary Application Modes

Assistant - Task-oriented systems that provide specialized help and execute complex procedures by leveraging domain knowledge and maintaining user context.

Workflow - Process-driven systems that orchestrate multi-step operations through structured execution paths and tool coordination.

Deep Research - Multi-source investigation systems that gather, synthesize, and report findings over extended periods. Think academic research assistants or market analysis agents working on complex, long-term inquiries.

Application Mode → Memory Type Mapping

Application Mode	Memory Types	Why These Types?
Assistant	Persona, Conversational, Semantic	Needs consistent personality, dialogue history, and domain knowledge (works for coding, legal, medical, etc.)
Workflow	Procedural, Toolbox,	Requires step-by-step processes, available tools, and execution state tracking

Workflow	Procedural, Toolbox, Workflow	Requires step-by-step processes, available tools, and execution state tracking
Deep Research	Episodic, Semantic, Working, Shared	Needs specific findings, general knowledge, active research state, and multi-agent coordination

#100DaysOfAgentMemory

Why Application Mode Matters

Understanding the application mode of your AI agent is not merely a design consideration—it's a foundational decision that cascades through every layer of your technical stack. The application mode determines the fundamental operational requirements of your agent, which in turn drives architectural decisions across four critical areas: architecture requirements, memory provider(database)optimization, embedding model selection and agentic architecture design

Memory Architecture Requirements

Different application modes require distinct memory type combinations. An Assistant mode agent requires persona memory for consistent character representation, conversational memory for dialogue continuity, and semantic memory for domain knowledge storage. In contrast, a Workflow mode agent prioritizes procedural memory for step-by-step processes, toolbox memory for available functions, and workflow memory for execution state tracking. Of course these are not prescriptive and you can find situations where agents acting as assistant require procedural memory as well.

Memory Provider Optimization

The retrieval methods and indexing strategies for your memory provider (database) must align with your application mode's access patterns. Assistant mode agents typically require fast semantic similarity searches for contextual responses, while Workflow mode agents need structured queries for process state retrieval and procedural lookup operations. So having a memory provider with various retrieval capabilities is one of the key solution requirement to consider when conducting vendor selection.

Embedding Model Selection

Application mode influences the choice between general-purpose and specialized

Embedding Model Selection

Application mode influences the choice between general-purpose and specialized embedding models. A general-purpose model like Voyage-3-large may suffice for broad Assistant mode applications, while domain-specific variants benefit from specialized models such as Voyage-Code-3 for development assistants or domain-fine-tuned models for legal or financial applications.

#100DaysOfAgentMemory

Agentic Architecture Design

The operational patterns of different application modes drive fundamental architectural decisions. Deep Research mode, for instance, typically requires multi-agent orchestration due to its long-running execution requirements and the need for query parallelization. In this architecture, specialized agents handle discrete research aspects, consolidate findings in shared memory systems, and coordinate through an orchestrator agent that synthesizes the final output. A good deep dive into what this agentic architecture looks like in practice is provided in Anthropic piece: "How we built our multi-agent research system"

Day 5/100 of Agent Memory

The best way to learn is to build.
So I'm building MemoRizz.

Five days into this journey, and I'm already exploring some fascinating research about agent memory architectures. But reading papers, articles and theorizing only gets you so far. To truly understand how memory works in AI agents, you need to get your hands dirty and built

Introducing MemoRizz
I'm building an open-source Python library that serves as a memory layer for AI applications. Think of it as the practical implementation of everything I'm learning in this hashtag#100DaysOfAgentMemory series.
I will probably get a lot of things(definitions, techniques and implementations) wrong, and I think that's fine.

Learning in Public, Building in Open
The point is that most of the concept I explore in this series gets tested, implemented, and refined in MemoRizz. When I discover something interesting about how procedural memory should work in workflow agents, it goes straight into the codebase.

On day 5 I worked on a form of episodic memories that some refer to as summarisation or observations. This is where you consolidate logs of interactions within a session, and compress them into summaries that can be used in future execution of agents for broader context of a problem domain or environment. A question I asked myself is when do I trigger a summarisation? When the context window gets full ? At the end of the session or day?

Either way I will release this feature in a few days in MemoRizz.

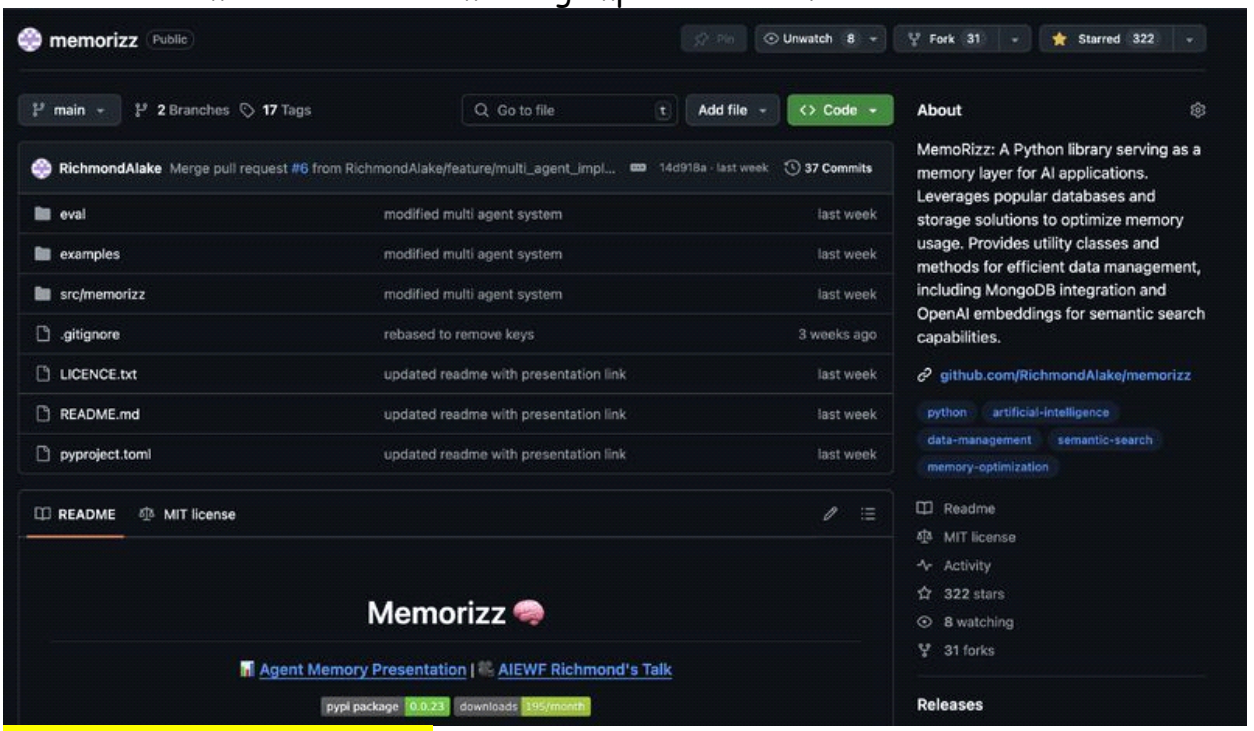
Why Open Source?

Agent memory is still an emerging field, and I believe the breakthroughs will come from the community—researchers, engineers, and builders working together to solve these fundamental challenges.

What's Next?

Over the next 95 days, MemoRizz will evolve alongside my understanding.
Each post in this series will likely spawn new features, optimizations, or architectural decisions in the library.
It's going to be messy and iterative, but we will a lot along the way.

The library is already gaining traction
—322 stars and growing
—which tells me we're onto something important here.



Day 6/100 of Agent Memory

Summarization, also called compression, compaction etc
Is one of the techniques that has maintained high relevance in AI application development. Anthropic refer to it in their latest piece on how to build multi agent systems effectively and pass information from one agent to another when context window are approaching limits.

So I wanted to dive deep into summaristion within Agent Memory.The best way to learn is to build. So I built memory summarization into MemoRizz. The key challenge is figuring out how to compress months of agent interactions without losing the essence of what makes each conversation valuable?

The Summarization Challenge

As agents accumulate conversations, they face context windows overflow, retrieval slows down, and important patterns get buried.
The solution? Intelligent memory summarization—a form of episodic memory that captures the essence of interactions while dramatically reducing storage footprint.

Key Questions I Grappled With:
→ When should summaries be generated? (After N interactions? Daily? Weekly?)
→ How often should this process run? (Real-time vs batch processing?)
→ How far back should we look when generating summaries? (Last day? Week? Month?)
→ What memory types should summaries update? (Should they influence persona?)

Design Philosophy: Developer Choice
Rather than imposing a rigid framework, I decided to give developers the tools and let them decide the strategy. Different applications have different needs—a customer service agent might need daily summaries, while a research assistant might work with weekly compilations.

Persona Evolution
I also experimented with letting summaries update the agent's persona—essentially allowing personality traits to evolve based on interaction patterns. Again, many approaches possible, many questions to consider:
→ Should personality changes be gradual or immediate?
→ Which interaction patterns should influence persona traits?

My solution: Give you the method, you decide the strategy.
The code is open source🔗: https://lnkd.in/ebFNZcdS

```
import MongoDBConfig, MongoDBProvider
from memorizz.memory_provider.mongodb.provider
from memorizz.persona import Persona, RoleType

# Create a memory provider
mongodb_config = MongoDBConfig(uri=os.environ["MONGODB_URI"])
memory_provider = MongoDBProvider(mongodb_config)


# Create a persona for our agent
persona = Persona(
    name="Alex",
    role=RoleType.ASSISTANT,
    goals="Help users learn and provide thoughtful assistance",
    background="An AI assistant focused on being helpful, educational, and adaptive"
)

# Create an agent with the persona
agent = MemAgent(
    model=OpenAI(model="gpt-4"),
```

```python
# Create an agent with the persona
agent = MemAgent(
    model=OpenAI(model="gpt-4"),
    persona=persona,
    instruction="You are a helpful learning assistant that adapts based on interactions",
    application_mode="assistant",
    memory_provider=memory_provider
)

# Generate summaries from the accumulated memories
summary_ids = agent.generate_summaries(
    days_back=1,  # Look back 1 day (covers all our interactions)
    max_memories_per_summary=5  # Smaller chunks for demo
)

# Update persona based on summaries
persona_updated = agent.update_persona_from_summaries(
    max_summaries=3,  # Use up to 3 most recent summaries
    save=True         # Save the updated persona
)
```

snapplify.com

Day 6/100 of Agent Memory 🤔

An entry point for a lot of AI Engineers into Agent Memory is through RAG.
But make no mistake
RAG is just the beginning and not the end of Agent memory.

So today, I want to cover all the relevant aspects of RAG you should be thinking about when it comes to memory engineering.
🔄 Data Preparation - We typically start here, where raw data become structured chunks. Think data sciencey type activties
📊 Embedding Generation - Converting data into semantic vectors that capture meaning, not just keywords
🗄 Multi-Index Storage - Supporting different memory access patterns (semantic, temporal, relational)
🔍 Hybrid Search - Because sometimes you need exact matches, sometimes semantic similarity and most time...both.
📈 Reranking - This surfaces up key memories above noise

But here's what most people miss:
The important aspect is Retrieval.
There are various hierarchies/types of Agent memory, and they all have their own retrieval requirements:
→ Episodic Memory - Needs temporal + semantic retrieval
→ Semantic Memory - Requires conceptual similarity search
→ Procedural Memory - Needs pattern-based retrieval
→ Working Memory - Requires immediate access retrieval

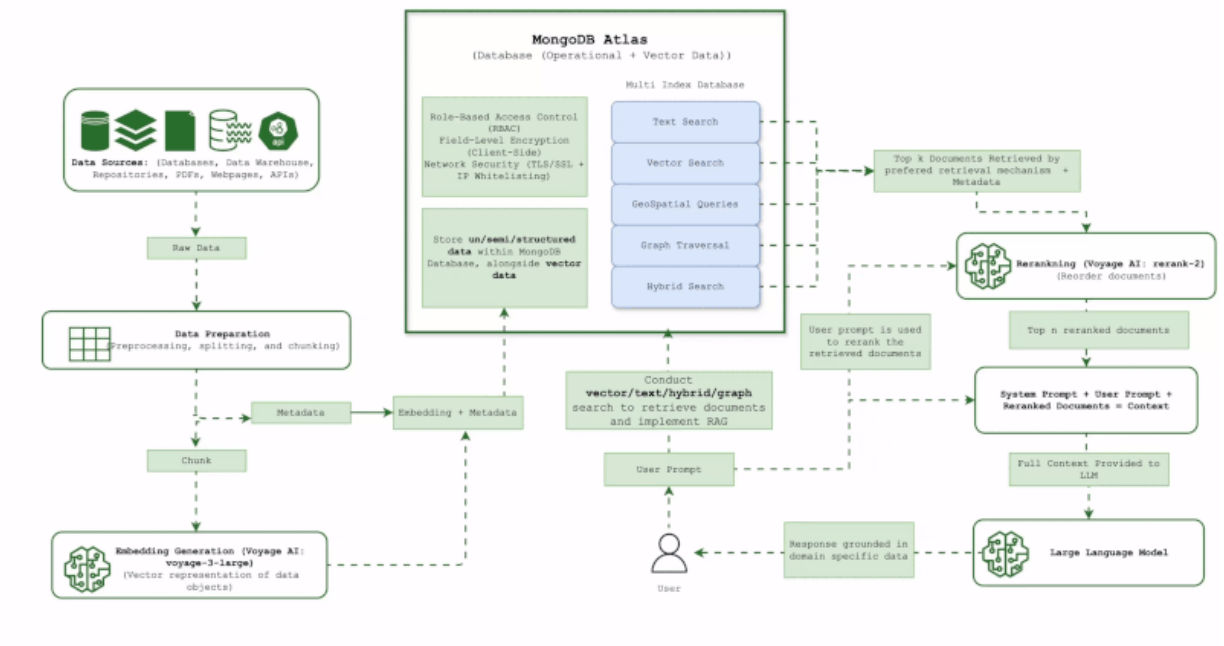
And lately, folks have been asking me why don't I use a database optimized solely for vector search or one that just focuses on graph traversal.
Here's the thing: I'd rather have one Swiss Army knife than carry around five different screwdrivers (this line is funny if you know how much I don't do DIY).
But you get the point.

One thing I find very helpful when implementing memory management solutions is to find solutions that consolidate as many features as possible.

In Agent memory, specialization isn't crucial
I don't need a specialized database for each memory type.
I need one that handles them all efficiently.

That's why MongoDB Atlas works so well for MemoRizz.

But I've built MemoRizz in a way that allows for you to place your own database as a memory provider.
Happy to see what people hack away with.



Day 7/100 of Agent Memory

You know we had to join the conversation
When Anthropic drops "Building Multi-Agent Research Systems" and Cognition says "Don't Build Multi-Agents," the AI world goes into debate mode.

But here's the plot twist
They're both saying the exact same thing.

Here's the TLDR ↴
Anthropic: "Multi-agents work great for deep research with sophisticated memory management"
Cognition: "Single agents are better for conversational tasks with continuous memory flow"

The Reality: Both teams arrived at the same core insight
Agents fail without robust memory management.

Anthropic chose to solve memory distribution across multiple agents, while Cognition chose memory consolidation within single agents.

Developers shouldn't be debating about single vs multi-agent architecture
That's like debating whether to use a hammer or screwdriver when you don't even have a toolbox. 🧰 (And yes, I am still sticking with DIY references, even though I am terrible at them (see day 6 if you don't get it..))

I have shared our perspective on building Agent in the piece below:
📎 https://lnkd.in/gJXDMVDB

Key Learnings:
✅ Application mode determines memory architecture
✅ Memory engineering is becoming its own specialization
✅ Systematic agent observation beats vibes-based debugging
✅ Context window is not enough you need persistent, retrievable memory

Day 8/100 of Agent Memory 🧠

I just realised
I'm probably in a bubble.
But if I'm not, then I'm in a privileged position.

This week, I realized that a lot of people are working/thinking/talking about Agent Memory. And not just the 100+ developers that have messaged/followed me after that talk (you know the one).

What I'm seeing:
🏢 Enterprise conversations - Leaders in large orgs building agentic systems, looking for guidance on how they can go beyond context engineering and move into memory engineering for AI agents

🚀 Startup creativity - Companies thinking more creatively than ever about applying agents to various industries. Their creativity is genuinely inspiring.

📊 Evaluation gaps - Spending time with Roie Schwaber-Cohen from Galileo as we prepare for our webinar on AI reliability and evals made me realize there's so much more that can be done around Agent memory evaluation.

Let's dive into this.

The Evaluation Challenge
Folks are already looking into this.
We have LongMemEval testing long-term memory capabilities, LoCoMo evaluating localization and memory, and emerging frameworks trying to measure memory persistence, retrieval accuracy, and forgetting patterns.

But we're still in the early innings of understanding how to properly evaluate agent memory systems. Most current benchmarks test retrieval accuracy, but what about memory formation quality? Memory prioritization? The ability to forget irrelevant information?

Evaluating Agent memory involves measuring performance of the retrieval, generative, and storage components of your AI system(and some aspects that I have not encountered yet).

I'll go more in the eval side of things in a few weeks.
This weekend I will be spending some time exploring libraries such as LangMem, MemGPT, Mem0 and see what **they** have in common, as well as differentiating factors.

Day 9/100 of Agent Memory 🧠

I usually say these two things ↴
1. I usually say vector search isn't all you need
2. I also say that human intelligence is one of the best blueprint we have for building AI. Mainly around the hierarchical understanding of information.

I came across a summary of a paper that combined 1 and 2 together.

This is a study from University of Geneva that validates how we've been building in Agent Memory systems. At at high level the research reveals that human memory chooses meaning over surface similarities (when familiar mental categories are available).

The Finding:
Memory favors structural connections when it can draw on familiar mental categories (like "excuses," "conflicts," "patterns")

It defaults to surface-level cues only when no such category exists
Structural memories reflect deeper understanding and enable better knowledge transfer.

Practical Scenario:
Imagine a customer service agent.
A customer says:
🙋 "I'm frustrated because your app keeps crashing during checkout."

1️⃣ Surface-level matching would retrieve:
Previous conversations about "app crashes" or "checkout issues"
2️⃣ Structural matching with mental categories would retrieve: Previous cases categorized as "user frustration due to system failure" - including a website login bug, a payment gateway timeout, and a mobile app freeze. Which seem different but have the same underlying problem pattern.

The agent with structural memory understanding can immediately apply solutions from the broader category of "system reliability issues causing user frustration" rather than just "app crash fixes." I think this also taps into works we are seeing with intent engineering, where we get the initial query from a user and detect intent and augment the query.

Why this matters for Agent Memory:
Most AI systems still rely on surface-level similarity (vector embeddings, keyword matching). But this research suggests we should be building agents that:
→ Develop conceptual categories from interactions over time
→ Prioritize structural patterns over surface matches when categories exist
→ Fall back to surface similarity only when conceptual understanding is lacking

Practical implications:
This is why simple vector search isn't enough.
Agent Memory systems need:
➡ Category formation mechanisms
➡ Pattern recognition across different contexts
➡ Adaptive encoding based on available mental models
➡ Hierarchical retrieval (conceptual first, surface second)

It's becoming obvious that the topic of Agent memory isn't just focused on storage and retrieval, but how we structure information, and the hierarchical organisation of data in storage and in the context windows of LLMs can contribute to an Agent's performance.

📎 Article: https://lnkd.in/eWkZyMgR

Day 10/100 of Agent Memory 🧠

You are going to be asked this question a lot ↴
"What are the signals that Agent Memory is important?"

Not everyone realizes that industry is closing the memory gap - and it's happening faster than expected. So today, there's some information that will answer the question above, and let you drive the next AI initiative in your org.

Paying attention to signals:
Industry momentum from incumbents, AI startups, and open-source software confirms that there's a significant attempt to close the "memory gap." In April 2025, OpenAI activated full-history recall for ChatGPT, letting the assistant automatically draw on and refine everything a user has ever shared (1).

Anthropic's May 2025 launch of Claude 4 introduced enhanced memory capabilities for maintaining context in long-horizon tasks (2), while Microsoft's spring 2025 Copilot updates introduced a personalized memory layer that stores tasks and preferences for context-aware assistance (3).

Taken together, these launches signal a sector-wide pivot from stateless chatbots toward agents that learn and evolve over time.

AI startups have also embraced memory as a core product feature, with notable examples including Cursor, the AI Code Editor created by Anysphere, which released a "Memories" feature in beta as part of their 1.0 launch in June 2025 (4).

Additionally, numerous open-source projects have emerged specifically to address memory management in agentic applications, including MemGPT, Zep, MemoryBank, Mem0 and others.

What this means for you:
The next wave of competitive advantage lies in understanding how to architect, implement, and optimize memory management systems that transform reactive applications into truly intelligent agents.

For those who love numbers:
The agentic AI market is projected to grow from an estimated $5.25 billion in 2024 to over $52 billion by 2030, with over 60% of new enterprise AI deployments in 2025 expected to include agentic capabilities, according to Gartner's 2025 Emerging Tech Report (5).

Whether you're building customer service bots that remember user preferences, coding assistants that learn from your patterns, or workflow automation that adapts to organizational processes, memory engineering is quickly becoming the differentiating factor between impressive demos and production-ready AI systems that users genuinely trust and rely upon.

Citations:
1. https://lnkd.in/eix9QKne
2. https://lnkd.in/e62vf49M
3. https://lnkd.in/eR4EfwUq
4. https://lnkd.in/eWHKYdJV
5. https://lnkd.in/eih3p86w

Day 11/100 of Agent Memory 🧠

Alright I brought it back to life.
I haven't posted on my YouTube channel in 2025.
Let's blow💨 the dust off the camera and get uploading.

And you guessed it
The topic here is
Agent Memory.

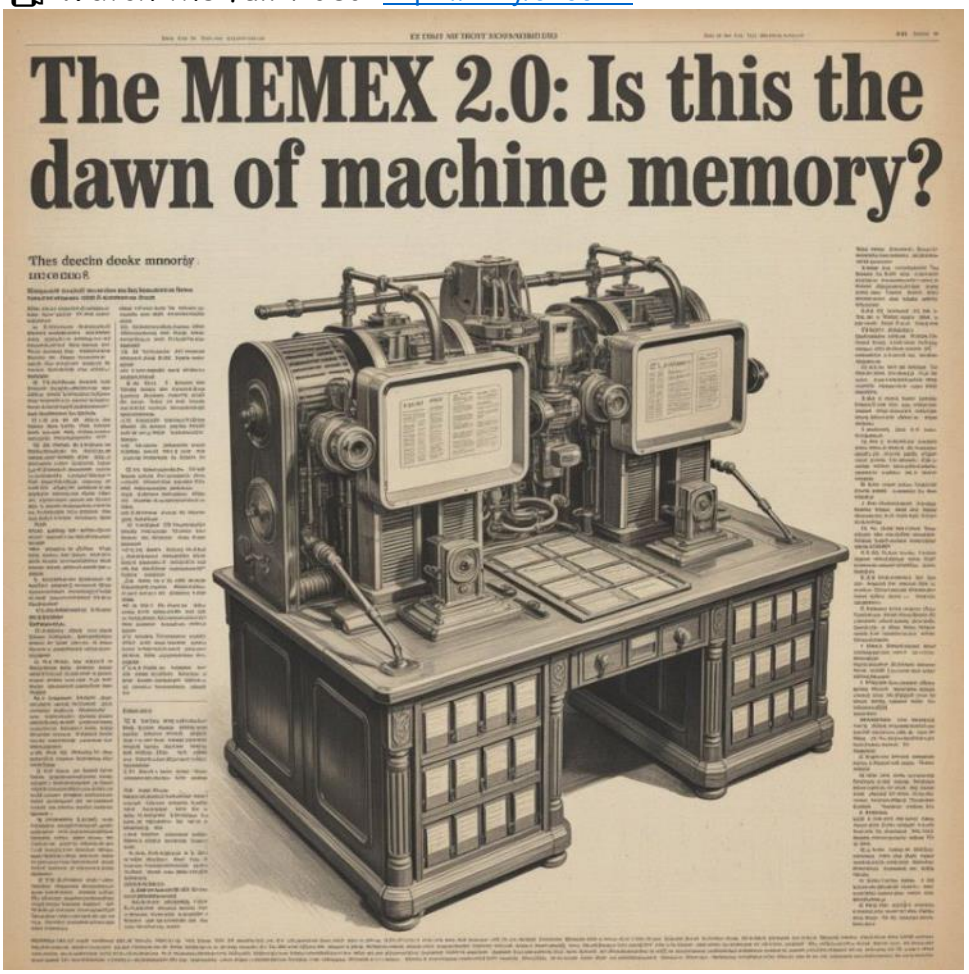
What you'll learn in this video:
In this deep dive, I explore how adding robust, persistent memory systems can transform agents from simple task executors to dynamic entities capable of managing complex workflows and interactions.

Key insights covered:
→ Latest research insights, including Anthropic's approach to memory in multi-agent systems
→ Memory engineering as a specialization within AI
→ Different types of memory: workflow memory and toolbox memory → How agents store, retrieve, and update knowledge efficiently
→ Memory-augmented agents revolutionizing AI reliability, believability, and capability

The future of AI agents isn't only about better reasoning - it's about agents that remember, learn, and adapt.

I unpack the potential of memory-augmented systems and why memory engineering is becoming the next frontier in AI Engineering.

📺 Watch the full video: https://bit.ly/3TJ3fxa



Day 12/100 of Agent Memory 🧠

The Agent Memory space is heating up
Just this week, I've had conversations with customers, reporters, professors and founders on Agent Memory in all various shapes and sizes.

One of the insightful conversations I had was with Daniel Porras Reyes from Flybridge to discuss Agent Memory and how MongoDB is the memory provider for AI Agents and Agentic Systems.

I would recommend reading the article linked below.

Key insight I shared:
Memory in AI isn't entirely new—concepts like semantic similarity and vector data have been around for years—but its application within modern AI agents is what's revolutionary. Agents are becoming prevalent in software, and the way we now use memory to enable personalization, learning, and adaptation in these systems represents a fresh paradigm shift.

MongoDB positions itself as a memory provider for agentic systems, transforming raw data into intelligent, retrievable knowledge through capabilities like with Voyage AI by MongoDB. We enable developers to build comprehensive memory management solutions with tools for graph, vector, text, and time-based queries—all optimized for low latency and production ready in one platform.

The big question:
That will be answered this year is
Where in the stack will memory value accrue?
Foundation models?
Agentic frameworks?
Database Systems?
Or specialized infrastructure providers?

The future:
Companies investing in robust memory systems today will gain user lock-in, compound intelligence, and operational efficiency.

📎Link: https://lnkd.in/ehKvP3gz

Day 13/100 of Agent Memory 🤩

This is the part I love.
And you can tell from the smile on my face.
Speaking to Founders and customers that are building in AI.

This week I had a conversation with Habtamu Asefa, where he showed me an educational personal assistant that utilizes memory as a personalization mechanism for students. The system goes beyond generic educational content.
It remembers how each student learns best, tracks their progress across subjects like Math and Physics, and adapts its teaching approach based on accumulated interaction patterns.

Personalization is everything in AI product features and development, especially as we move toward applications that can act on our behalf. And memory is what makes personalization possible. When AI applications remember user behavior patterns - like learning preferences, communication styles, task priorities, or decision-making tendencies - they can pass this rich contextual information to the LLM. Instead of generic responses, you get outputs tailored to how you specifically work and think.

The memory components working together can look like this ↴:
→ Preference evolution tracked over time
→ Behavioral patterns stored in semantic memory

→ Interaction history captured in episodic memory
→ Context-aware retrieval that surfaces the right memories for each situation

Seeing memory engineering being applied to solve real problems in emerging markets. This is where AI agents with persistent memory will have transformational impact. And I was really impressed with the tool and how much impact it could have in the education sector in Ethiopia.

On a different note, I am thinking of starting a Discord to start more constructive discussions around Agent Memory, I see a lot of people building interesting things.

As AI systems evolve from stateless LLMs to **autonomous agents**, memory becomes a defining component of intelligence.

In human cognition, memory is multifaceted from episodic and semantic memory to working and procedural memory. Similarly, AI agents require **structured memory systems** to perform reliably across time, tasks, and modalities.

In agentic architectures, memory enables:

- **Contextual persistence** beyond the LLM's token window
- **Behavioral adaptation** based on prior interactions
- **Knowledge accumulation** across sessions
- **Dynamic planning** in open-ended environments

Key memory types include:

- **Working Memory**: short-term context (LLM token window)
- **Long-Term Memory**: entity-level and episodic histories
- **Knowledge Stores**: structured and unstructured world knowledge

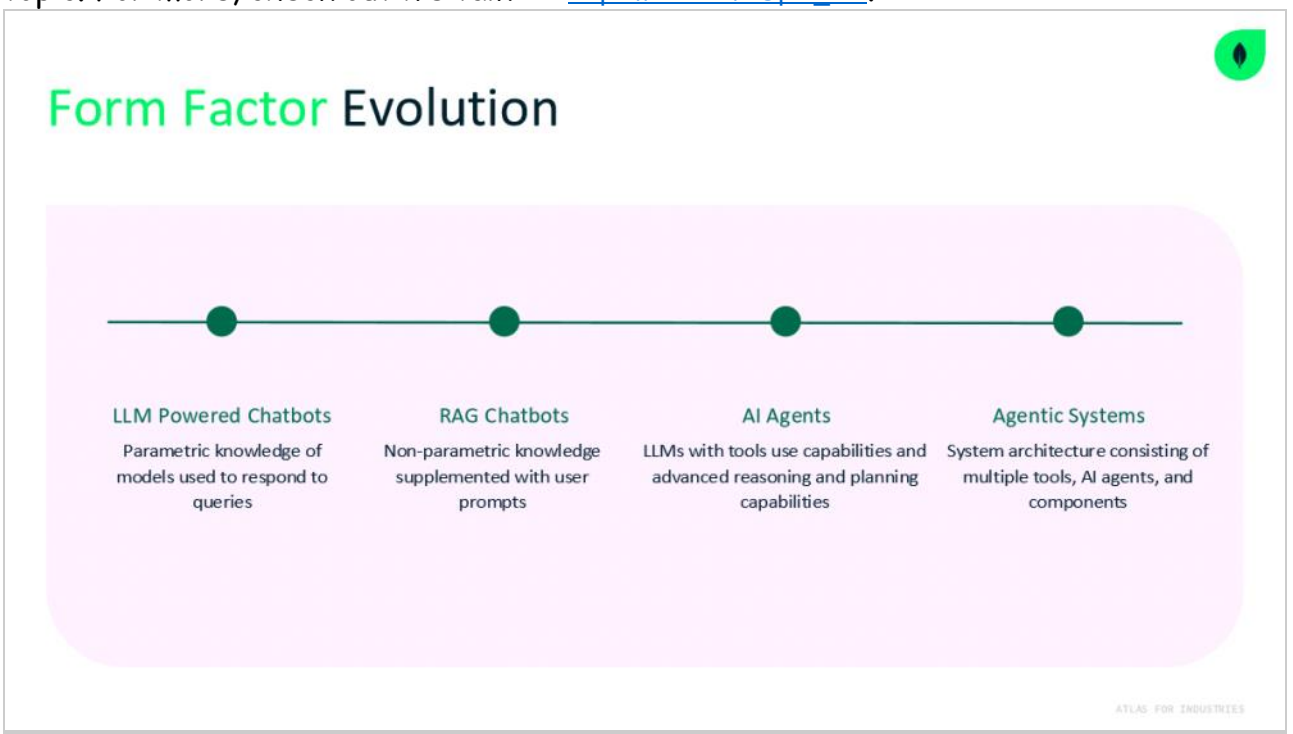
Effective memory management integrates:

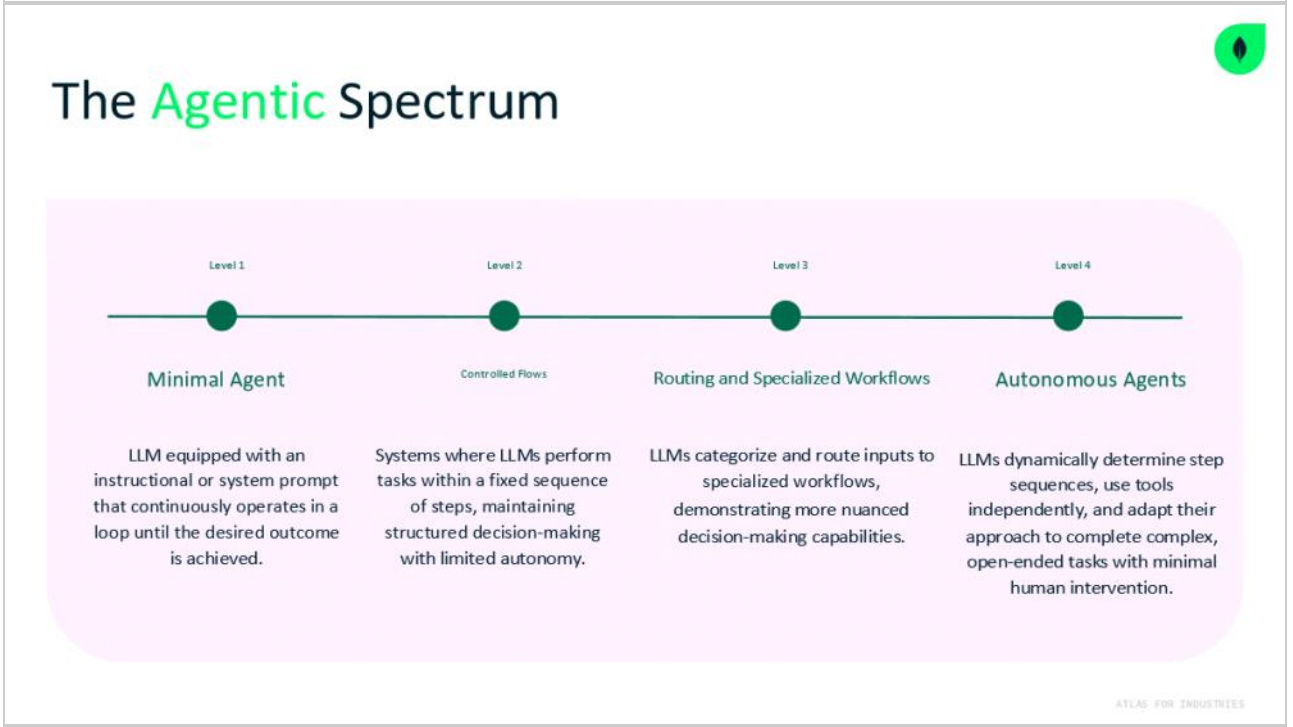
- Vector and hybrid search
- Retrieval-augmented generation (RAG)
- Embedding models & rerankers
- Systematic memory orchestration across APIs, databases, and inputs

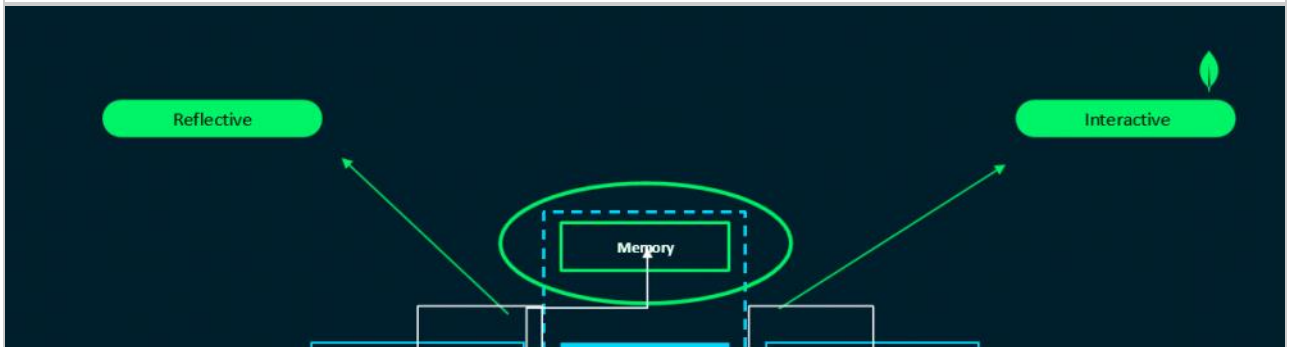
MongoDB is the memory provider for agentic systems, when enhanced with these capabilities, will become core memory layers too help agents retain context, recall knowledge, and make informed decisions across sessions.

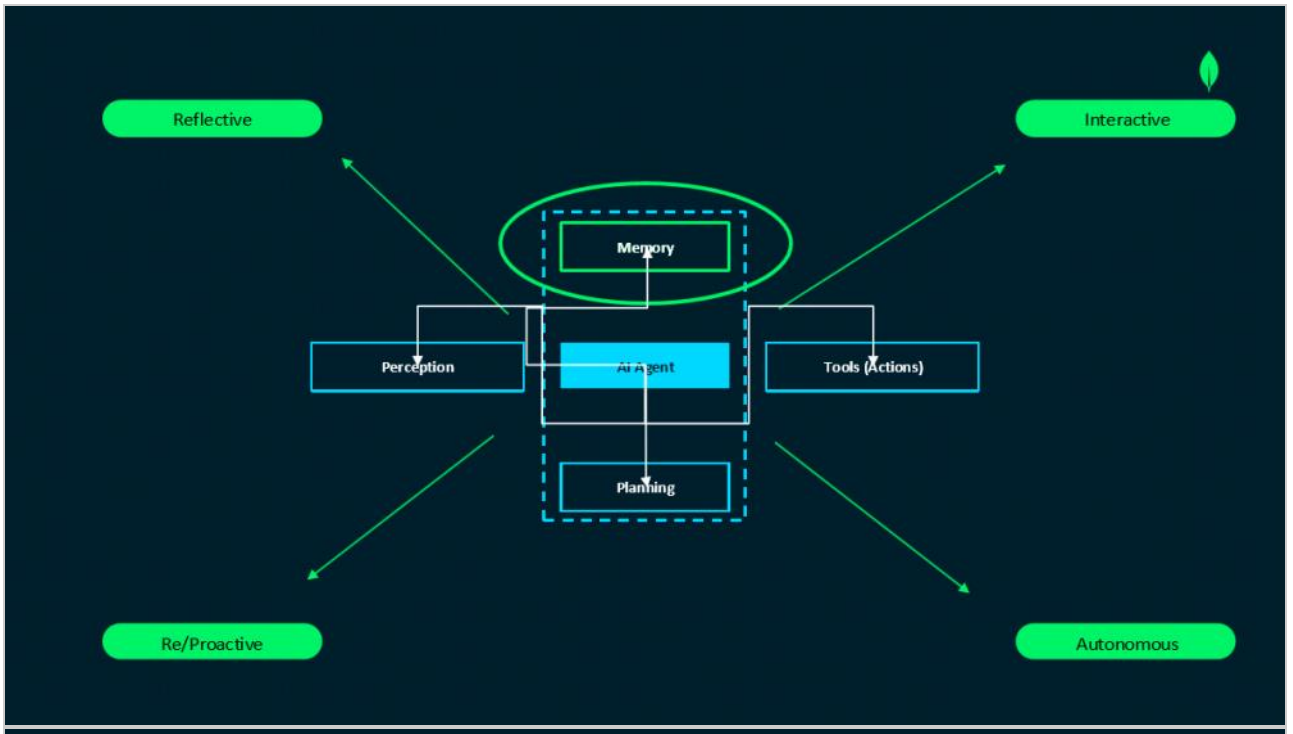
AI agents that remember are capable, believable, and increasingly autonomous.

Special thanks to Richmond Alake for his insights and perspective, a great guide on this topic. For more, check out his talk: -- https://lnkd.in/eUpD_cnf.



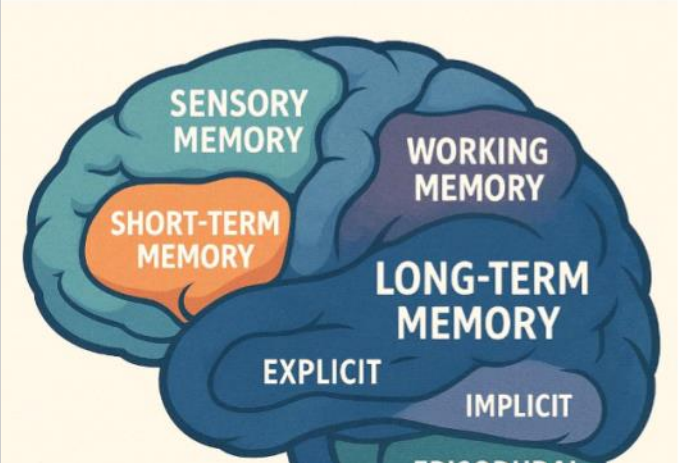






Artificial Intelligence is the scientific endeavor to create a computational form of **intelligence distinct from organic intelligence**, one that convincingly **mimics human cognitive abilities**.

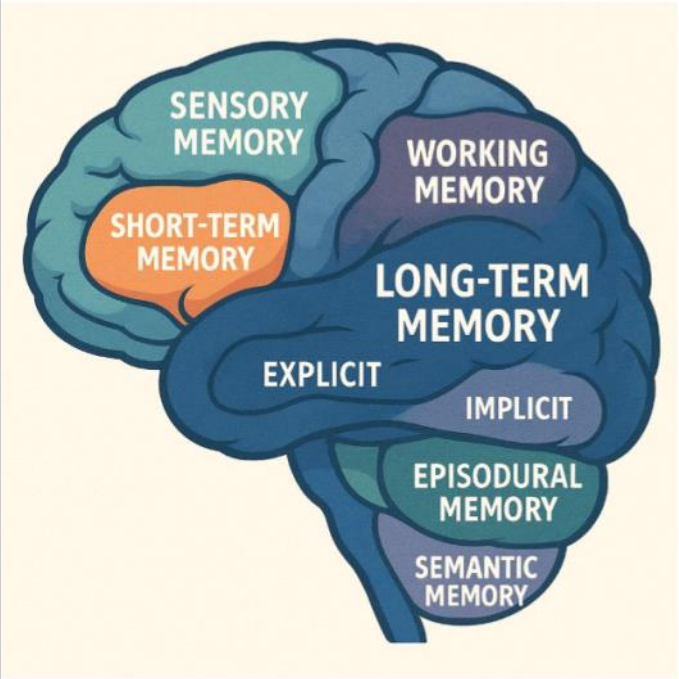
Artificial General Intelligence (AGI) refers to artificial intelligence that **surpasses human performance across most tasks** traditionally considered solvable by human intelligence.



The most effective form of intelligence—for now—is human intelligence, and **human memory capabilities substantially define intelligence**.

Examples of human memory include:

Sensory memory
Long-term memory
Working memory



The most effective form of intelligence—for now—is human intelligence, and **human memory capabilities substantially define intelligence**.

Examples of human memory include:

Sensory memory
Long-term memory
Working memory
Semantic memory
Episodic memory
Procedural memory and more

The Memory Provider For Agentic Systems: MongoDB

ATLAS FOR INDUSTRIES

Agent Memory

What is Agent Memory?

AI agent memory is the **persistent cognitive architecture** that allows agents to **accumulate** knowledge, **maintain** contextual awareness, and **adapt** their behavior based on historical interactions and learned experiences.

Memory makes Agents: Reliable, Believable and Capable

Agent Memory

What is Agent Memory?

AI agent memory is the **persistent cognitive architecture** that allows agents to **accumulate** knowledge, **maintain** contextual awareness, and **adapt** their behavior based on historical interactions and learned experiences.

Memory makes Agents: Reliable, Believable and Capable

Memory Management

What is Memory Management?

Memory management in agentic systems refers to the **systematic organization, persistence, and retrieval** of different types of information that AI agents need to function effectively across interactions and sessions.

1. Generation
2. Storage
3. Retrieval
4. Integration

Core Components of Agent Memory Management

1. Generation
2. Storage
3. Retrieval
4. Integration
5. Updating
6. Deletion

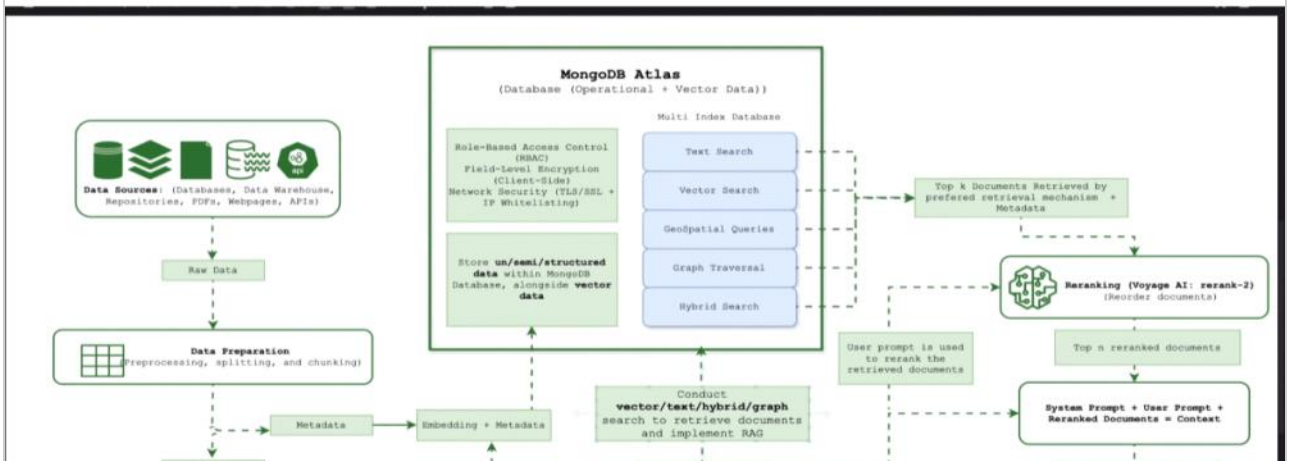
Core Components of Agent Memory Management

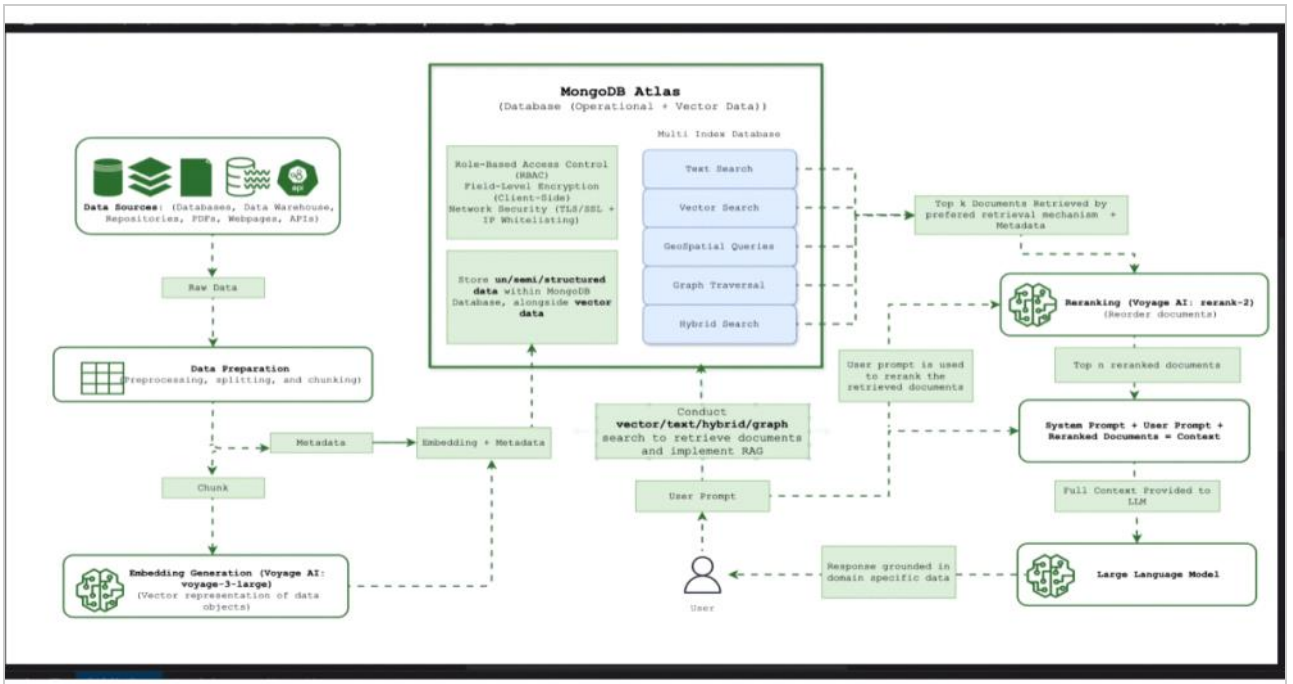
MONGODB AI CENTER OF EXCELLENCE

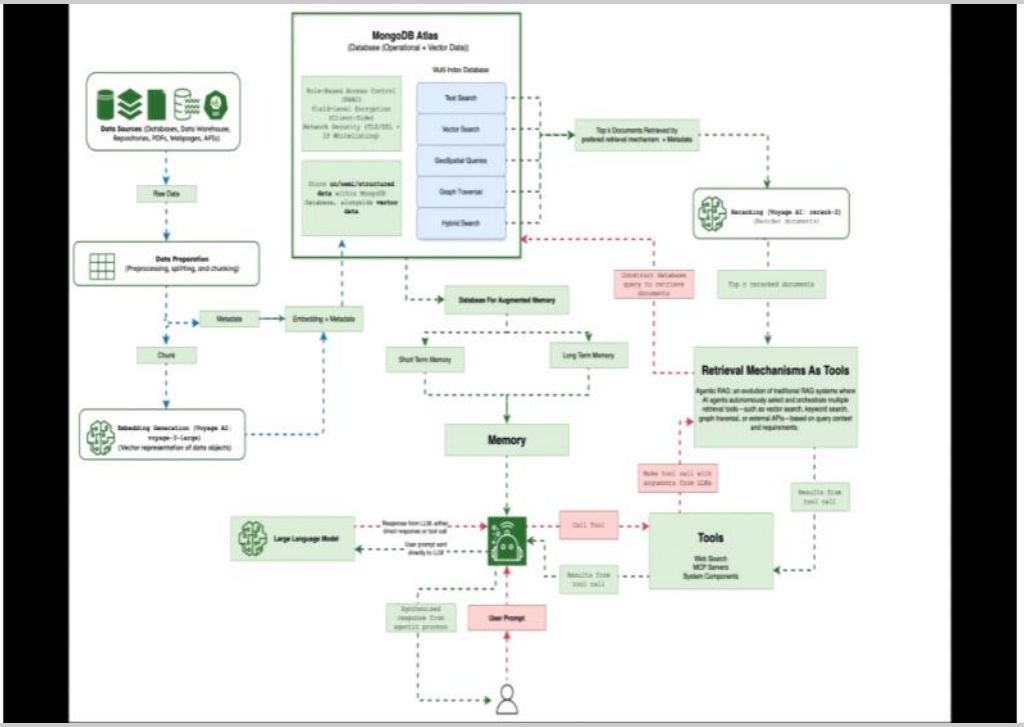
1. Generation
2. Storage
3. **Retrieval**
4. Integration
5. Updating
6. ~~Deletion~~ Forgetting

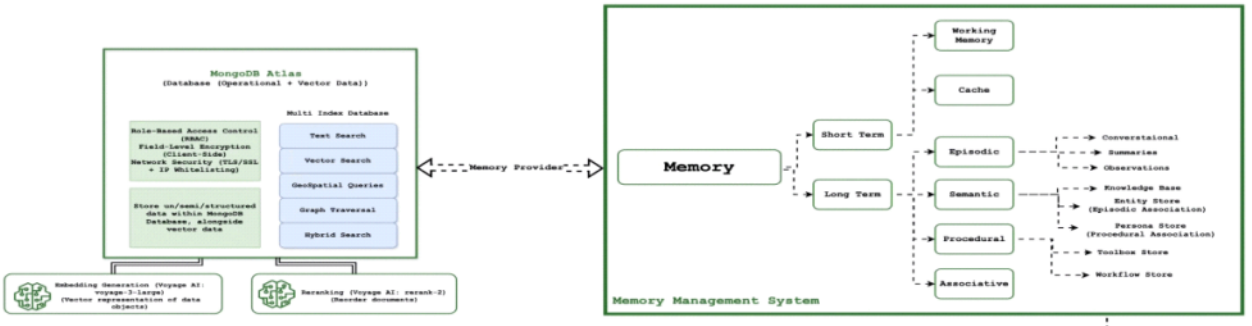
Core Components of Agent Memory Management

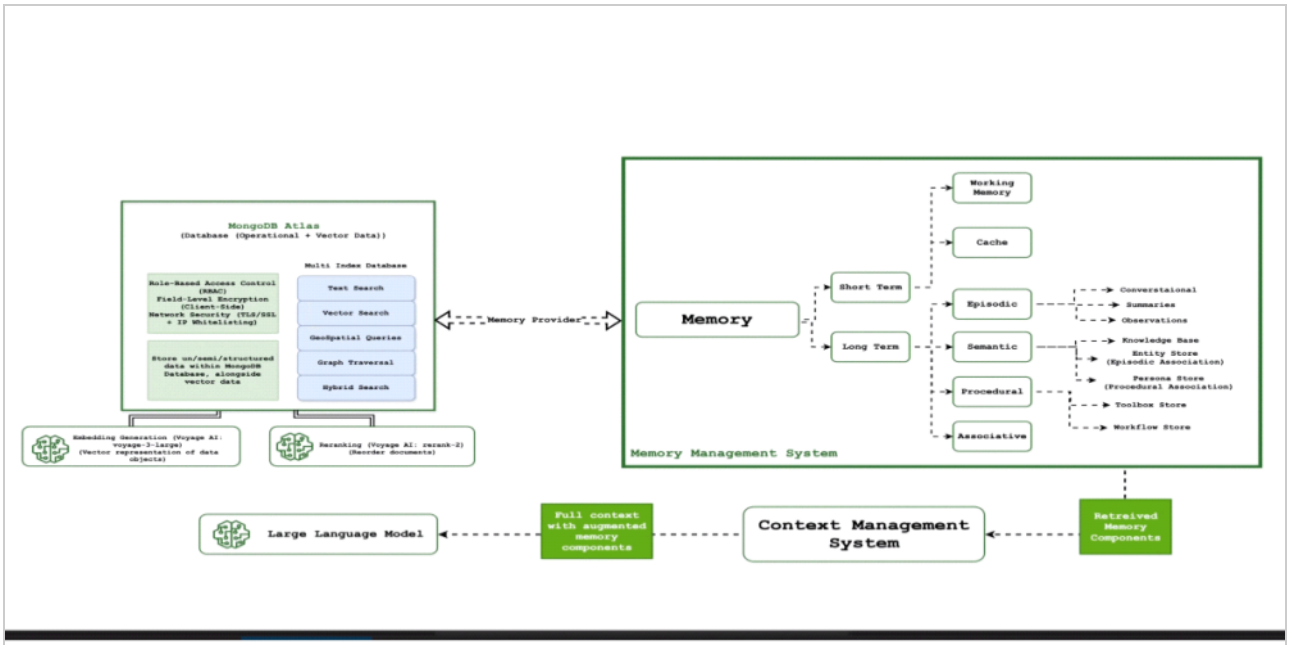
MONGODB AI CENTER OF EXCELLENCE

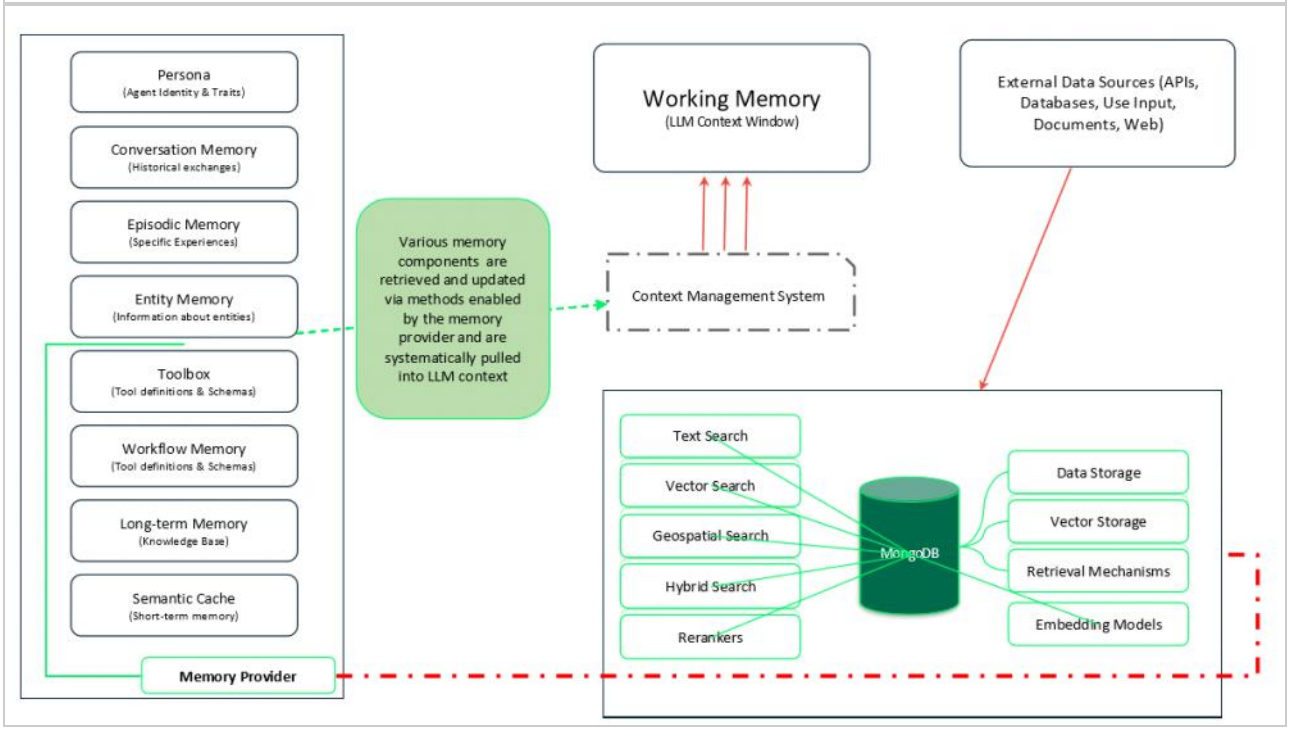



MongoDB Atlas
(Database (Operational + Vector Data))
Multi Index Database
Role-Based Access Control (RBAC)
Field-Level Encryption (Client-Side)
Network Security (TLS/SSL + IP Whitelisting)
Store un/semi/structured data within MongoDB Database, alongside vector data
Text Search
Vector Search
GeoSpatial Queries
Graph Traversal
Hybrid Search
Data Sources: (Databases, Data Warehouse, Repositories, PDFs, Webpages, APIs)
Raw Data
Data Preparation
(Preprocessing, splitting, and chunking)
Metadata
Embedding + Metadata
Chunk
Embedding Generation (Voyage AI: voyage-3-large)
(Vector representation of data objects)
Conduct vector/text/hybrid/graph search to retrieve documents and implement RAG
User Prompt
User
Top k Documents Retrieved by prefered retrieval mechanism + Metadata
Reranking (Voyage AI: rerank-2)
(Reorder documents)
User prompt is used to rerank the retrieved documents
Top n reranked documents
System Prompt + User Prompt + Reranked Documents = Context
Full Context Provided to LLM
Large Language Model
Response grounded in domain specific data


MongoDB Atlas
Memory
Retrieval Mechanisms As Tools
Tools
Large Language Model
User Prompt


MongoDB Atlas
(Database (Operational + Vector Data))
Multi Index Database
Role-Based Access Control (RBAC)
Field-Level Encryption (Client-Side)
Network Security (TLS/SSL + IP Whitelisting)
Store un/semi/structured data within MongoDB Database, alongside vector data
Text Search
Vector Search
GeoSpatial Queries
Graph Traversal
Hybrid Search
Memory Provider
Memory
Working Memory
Cache
Short Term
Long Term
Episodic
Semantic
Procedural
Associative
Conversational
Summaries
Observations
Knowledge Base
Entity Store (Episodic Association)
Persona Store (Procedural Association)
Toolbox Store
Workflow Store
Memory Management System





Day 14/100 of Agent Memory 🧠

Think of this as a Spotify playlist
- but for research papers

I've been asked lately what papers should I read to get clued on Agent Memory. Below are some of the papers that I think are worth spending some time on.

📚 Essential Agent Memory Reading List:

1. Generative Agents: Interactive Simulacra of Human Behavior The foundational paper that started it all for me really. More specifically the simple algorithm the researchers use to update memory in the agents is what insipored my exploration into how memory evolves from input from environment.
📎 https://lnkd.in/eHTFbb7N

2. MemGPT: Towards LLMs as Operating Systems OS-inspired memory management for LLMs. Uses virtual memory concepts to manage context windows intelligently through hierarchical memory tiers. Game-changing approach to context limitations, .
📎 https://lnkd.in/epmKySE7

3. A-MEM: Agentic Memory for LLM Agents Zettelkasten-inspired memory system that dynamically creates interconnected knowledge networks. Agents actively build and organize their own memory structures - truly agentic memory. 📎
https://lnkd.in/emY4tnsX

4. MemoryBank: Enhancing Large Language Models with Long-Term Memory Incorporates the Ebbinghaus Forgetting Curve to create human-like memory decay. Enables agents to selectively forget and reinforce memories based on importance and time - very elegant.
📎 https://lnkd.in/eU-xsnUK

5. ZEP: A Temporal Knowledge Graph Architecture for Agent Memory State-of-the-art temporal knowledge graph for agent memory.
📎 https://lnkd.in/e5AEQPaj

6. Agent Workflow Memory Focuses on procedural memory for web navigation tasks. Agents learn reusable workflows from past experiences and apply them to new tasks. 24.6% improvement on complex web tasks. I use this teqchnqiue in the library MemoRizz
📎 https://lnkd.in/e7iGVDHi

7. From Human Memory to AI Memory: A Survey on Memory Mechanisms in the Era of LLMs Comprehensive survey connecting human memory research to AI memory systems. Essential reading for understanding the theoretical foundations and mapping between biological and artificial memory. I read this extensively before my talk at the AI Engineer conference 📎 https://lnkd.in/ehvgr7Mp
Day 15/100 of Agent Memory 🧠
If you are reading this
Then congrats you are early.

The current focus in AI is on context engineering, but I don't think this will last.

I am currently exploring the distinction between prompt engineering, context engineering, and memory engineering. One is definitely a step up in both complexity and return on value.

The Evolution of AI Engineering:
🔗 Prompt Engineering - Input Optimization Static instruction crafting. "How do I get the LLM to respond correctly to this specific query?" Limited scope, immediate results, but no persistence.

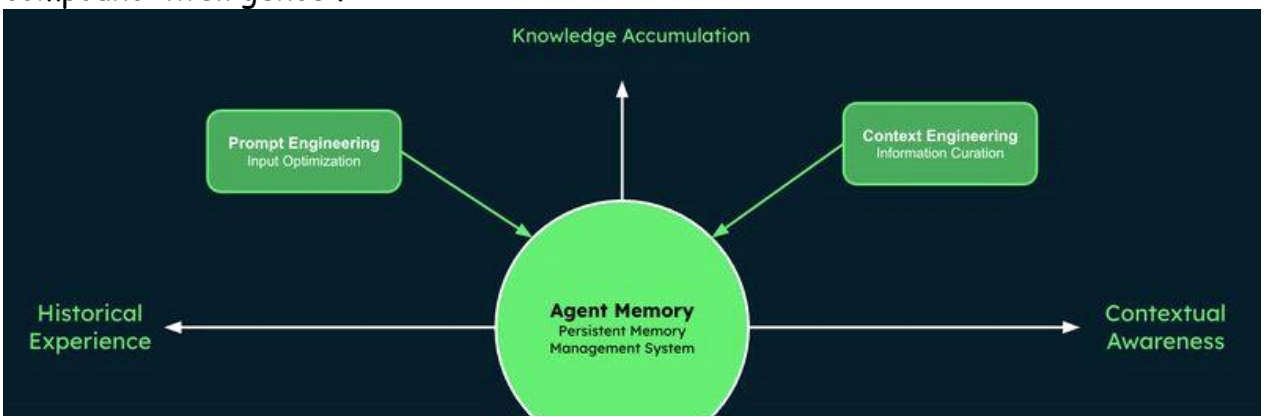
⚙️ Context Engineering - Information Curation
Dynamic context management. "How do I maintain relevant information across a conversation?" Better than prompting, but still session-bound.

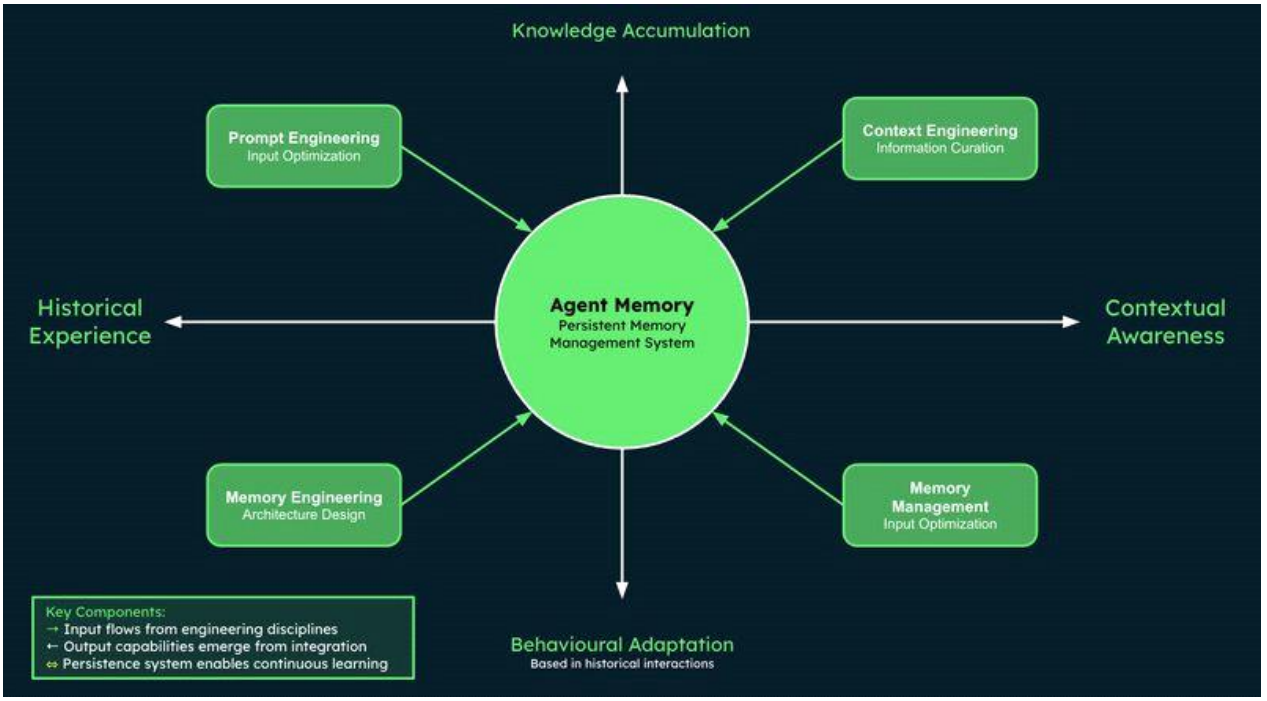
🧠 Memory Engineering - Architecture Design Persistent memory systems. "How do I build agents that learn, adapt, and remember across time?" The highest complexity, but exponential return on value.

Why context engineering won't last
Context engineering is the current "hot thing" because it feels like the natural(and better) evolution from prompt engineering. But it's still fundamentally limited - you can curate context perfectly, but without persistent memory, you're rebuilding intelligence from scratch every session.

Agent Memory sits at the center, integrating inputs from all AI engineering disciplines and outputs capabilities that none can achieve alone:
→ Knowledge Accumulation (learns over time)
→ Contextual Awareness (understands situations)
→ Behavioral Adaptation (changes based on experience)

While everyone's optimizing context windows, the smart money is already moving to persistent memory systems.
The companies building memory-first architectures today will have agents that compound intelligence .





Day 16/100 of Agent Memory 🥹

A picture speaks a thousand words
But in this case it does and doesn't
Because those thousand words just got wiped from memory.
Or at the very least, it's time to open a new chat.

If you are an extensive claude user, you definitely have hit this message ↴
"Claude hit the maximum length for this conversation. Please start a new conversation to continue chatting with Claude."

This screenshot attached to this post is beautifully ironic.
It perfectly illustrates the problem while simultaneously being the problem.

What just happened:
The relationship we built? Reset ▯
The accumulated understanding? Wiped 🧹
All our previous conversation context? Gone 👋

This is the "Goldfish Problem" I've been talking about for 16 days.
Even with million-token context windows, you will eventually hit the limit. Then what? Start over. Lose everything. The picture speaks volumes about memory limitations, but ironically demonstrates why context windows can't hold those volumes.

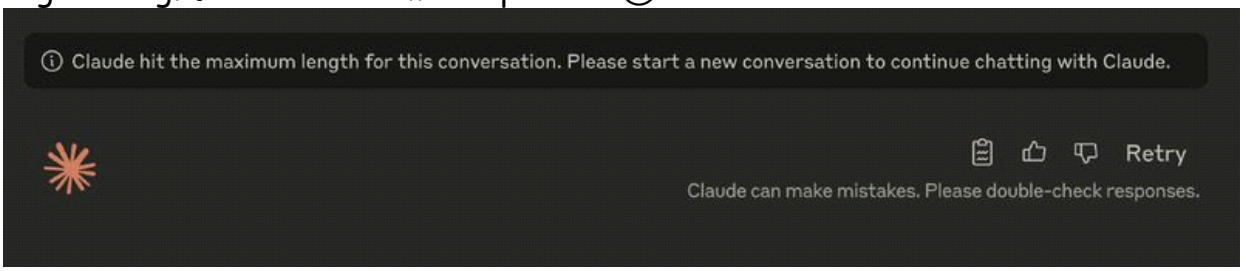
Agent Memory solves this:
→ Working memory maintains immediate context

→ Episodic memory preserves our conversation history
→ Semantic memory extracts key insights and preferences
→ Procedural memory remembers what approaches worked

Instead of "Please start a new conversation," imagine: "I remember our previous discussion about holiday destinations in Asia. Based on your preferences for beaches and vegan food, let me continue where we left off..."

Anthropic will probably fix this problem in a few months.

Next time someone ask why do we need to focus on Agent memory or memory engineering. Just send them this picture 😉



Day 17/100 of Agent Memory 🤣

Wait...the domain name memories.ai has been available all this time.
Maybe they bought it.

Either way, this at first glance looks pretty cool
Going to try it out this weekend.
There is no AGI, without memory.

> Today we're launching Memories.ai: we built the world's first Large Visual Memory Model — to give AI human-like visual memory.
>
> Why?
>
> Because AI is still mostly chat.
>
> But humans don't operate through chat.
>
> We see.
>
> We remember.
>
> We contextualize the world visually
>
> Today's AI has goldfish memory.
>
> Even the best video LLMs (like Gemini) max out at ~1 hour.

Even the best video LLMs (like Gemini) max out at ~1 hour.

Memories.ai has an unlimited context window for video understanding.

It sees, remembers, and reasons across entire video timelines — with ultra-low hallucination rates.

How does it work?

Think of it as the long-term visual memory of an AI — a deeply compressed, indexed, and retrievable store of visual experience, much like how we recall moments from the past.

Memories.ai watches, stores, and recalls it with pixel-perfect precision — on demand.

Works with your apps, agents, robots — you name it.

We believe Visual Memory is as fundamental as language models. That's why we're all in.

You would be surprised how many people don't understand the difference between LLM Memory vs Agent Memory.

Day 18/100 of Agent Memory 🧠

Today I will be touching on LLM Memory vs Agent Memory
Mainly, understanding the Fundamental Difference

When exploring memory in AI applications today, we encounter terms like LLM memory, Agent memory, and AI memory.

🔗 **LLM Memory (Internal to the model)**:
1 Parametric Memory (Weights): Knowledge baked into network parameters during training. Static during inference - only changes with fine-tuning.
2 Contextual Memory (Context Window): Transient session information within token limits. Lost when sessions end or tokens get truncated.

❌ **Limitations**:
→ Stateless inference
→ Bounded context window
→ No cross-session continuity

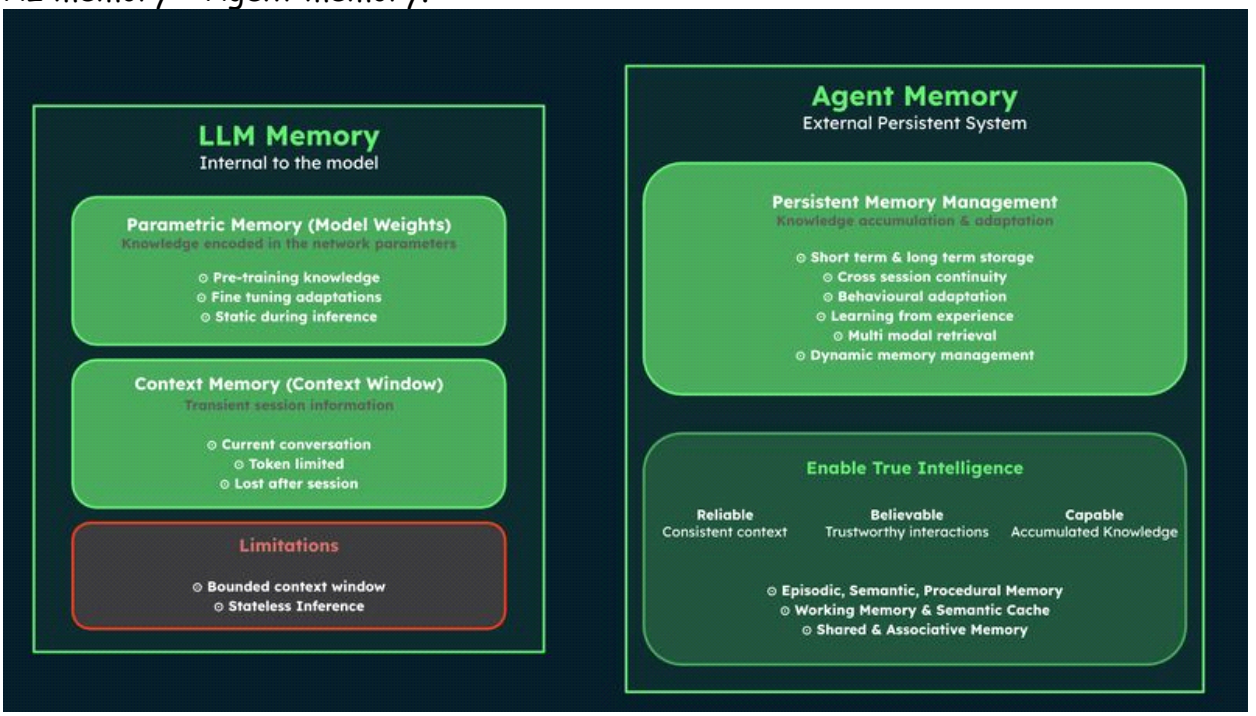
🧠 **Agent Memory (External Persistent System)**:

Persistent Memory Management: Knowledge accumulation & behavioral adaptation across sessions.
✅ **Capabilities**:
→ Dynamic memory management
→ Short & long-term storage
→ Cross-session continuity
→ Behavioral adaptation
→ Multi-modal retrieval

Key Insight: LLM memory isn't Agent memory. An LLM is just the cognitive engine of a broader AI agent. For true intelligence, you need external persistent systems that go far beyond basic retrieval. A form of exocortex that makes the Agent stateful.

An agent without augmented memory is merely a reflex agent - reacting only to current input without learning from previous iterations.

As software becomes increasingly agentic
AI memory = Agent memory.



Day 19/100 of Agent Memory 🧠

The Paradox of Perfect Memory: When It Works, Nobody Notices
I've been thinking about the best way to demo Agent Memory.
An obvious demo would be an agent without memory and one with.
The question to answer is what use case actually tells an impactful story.

Then I realized that although a lot of work goes into memory engineering in agents

and making our systems more reliable, believable and capable - when it all works perfectly, it becomes table stakes.

But when it doesn't... all hell breaks loose.

When memory works well:
→ Users don't even think about it
→ Conversations flow naturally
→ Agents seem "smart" and "intuitive"
→ It's just... expected

When memory fails:
→ "Why doesn't it remember what I said 5 minutes ago?"
→ "I already told you my preferences!"
→ "This feels like talking to a broken robot"
→ Users lose trust immediately

For those of you building consumer-facing agents: You might spend hours and days thinking about agent memory and implementing different memory engineering techniques for your customers, perfecting episodic recall, optimizing semantic retrieval, debugging procedural workflows...
Your users will never praise you for "excellent memory management."
They'll only notice when it breaks.

For Developers out there that are about to do a presentation or demo on why Agent memory matters↴
The best Agent Memory demos aren't about showing what memory can do - they're about showing what happens without it.
Show the chaos,
The frustration
The breakdown of trust.
Then show the alternative.

For day 20 - 30, we will get a bit more technical and start sharing some code and design patterns for agent memory.

Day 20/100 of Agent Memory 🧠
Memory engineering goes beyond just vector search and similarity scores.
Some of the implementation challenges that memory engineering will have to solve is how memory units are retrieved and updated.

Quick aside ↴
Memory units are structured data objects with grouped attributes that capture

discrete instances of information from user interactions, assistant responses, knowledge bases, workflow execution, and agent coordination, or simply kept...anything coming in and out of the context window.

But back to it
Developers will have to go beyond vector search similarity scores to determine how the relevant memory units are retrieved.

One of the first papers that provided a robust approach to this challenge is the Generative Agents paper by Park et al. from Stanford and Google, which introduced a weighted retrieval mechanism for memory stream access.

The Retrieval Function:
score = α·recency + β·importance + γ·relevance

🎯 **Relevance** → Cosine similarity between query embedding and memory embedding vectors. Standard semantic similarity computation using pre-trained embedding models.

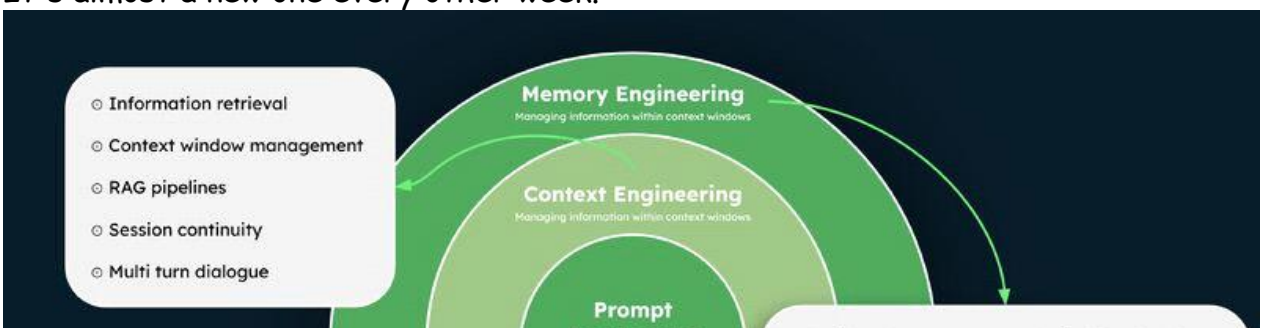
☆ **Importance** →Park et al. used LLM-assigned importance scores [1-10]. In my MemoRizz implementation, I compute this as the number of memory units that are related to the current memory unit. There are a variety of ways to approach computing this component of the retrieval function.

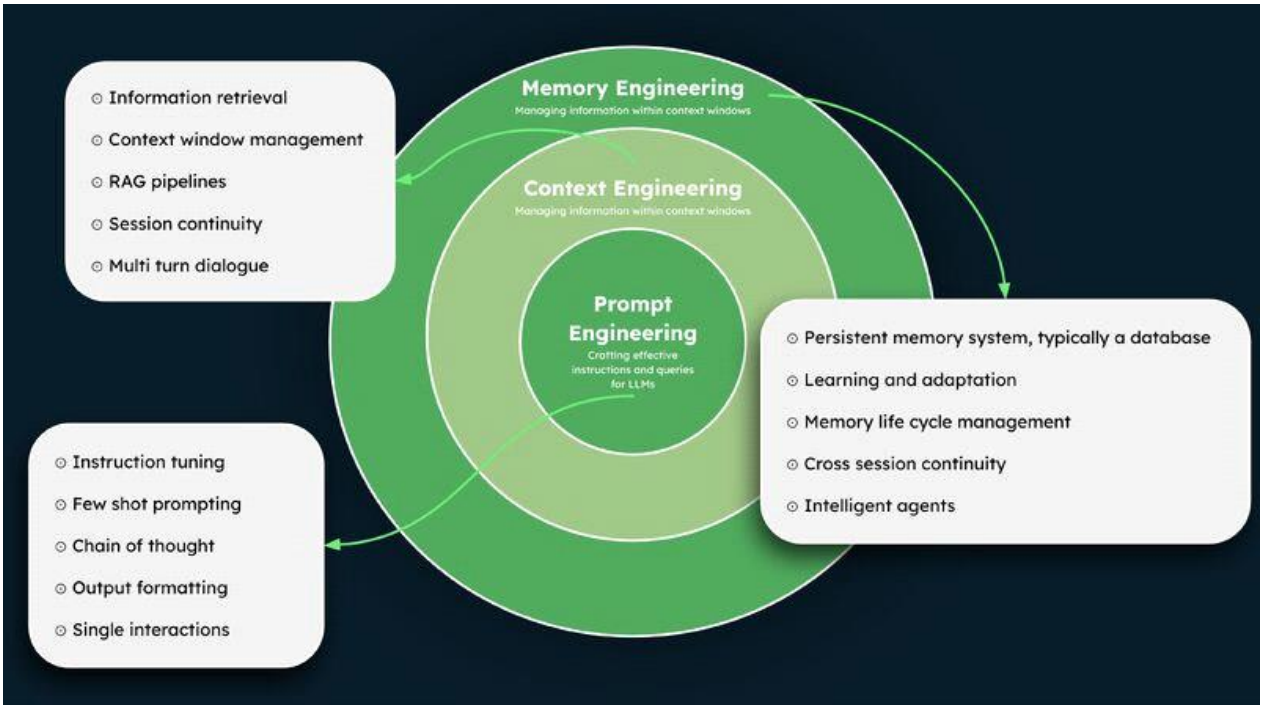
⏲ **Recency** → Time-based decay function. Recently accessed memories get boosted, while older ones gradually fade

We are moving memory retrieval from simple nearest-neighbor search into contextually-aware cognitive modeling - the foundation of sophisticated memory engineering architectures.

Basically memory engineering > simple vector databases.
Paper Link: https://lnkd.in/eVhhXA6x

As a side note, the increase in agent memory solution being released is impressive. It's almost a new one every other week.





Day 21/100 of Agent Memory 🧠

Today I took some time to deep dive into Context Engineering (And Why We Need to Move Beyond It).

I am putting together my thought for a piece that introduces memory engineering and so far, context engineering sits beautifully between prompt engineering and memory engineering.
It handles:
→ Dynamic information curation
→ Information retrieval and RAG pipelines
→ Session continuity and multi-turn dialogue
→ Context window management and optimization

It's definitely more sophisticated than basic prompt engineering (instruction tuning, few-shot prompting, chain of thought). You're actually managing information flow rather than just crafting better instructions.

But here's the problem: Context engineering is still fundamentally session-bound. You can perfectly curate context, optimize retrieval, manage conversations brilliantly... and then the session ends.

Memory Engineering is where the real work is, and I've spoken to a number of teams that are going beyond engieering what goes into the context and now, using what comes out the context window to update and create new memory units within the external persitence store which is an Agent memory component:

Memory engineering is about↴
→ Memory lifecycle management (when to remember, when to forget)
→ Persistent memory systems (databases that actually remember)
→ Cross-session continuity that survives context window limits
→ Learning and adaptation that compounds over time
→ True intelligent agents that evolve

I used the following resources to understand more on context engineering
📎 https://lnkd.in/ekwBSdQZ
📎 https://lnkd.in/emAPs83w
📎 https://lnkd.in/e4uhSu-3
📎 https://lnkd.in/eZ494iK9
📎 https://lnkd.in/eSfAyy6t
📎 https://lnkd.in/ektAFmbk
📎 https://lnkd.in/e7qhuDvZ

Pro tip: I dumped all these resources into NotebookLM with a customized prompt asking for technical deep-dives and implementation details. Generated an overview that got me conversation-ready in minutes. If you want to understand context engineering quickly, that's your shortcut.

Day 22/100 of Agent Memory 🧠

I keep talking to Founders and Developers working on Agent Memory
And it remains one of the best parts of exploring this space of AI.

So let me share a bit of the conversation I had with Arturo Arguello, CEO at Align Artificial Brains. He showed me a fascinating demo of Artificial Brain (https://lnkd.in/e9TxdwCV) and how he's implementing the Agent Memory concepts we've been exploring over the past few weeks now. I actually met Arturo in person in the hackathon MongoDB did at the AGI House in SF.

What blew me away:
Arturo has built a sophisticated knowledge graph visualization that shows associative memory in real-time. You can literally see how memories connect, form clusters, and influence each other.

In one of the post for the Agent Memories series, I think it was Day 19 or so, I mentioned that the best Agent memory demos are ones were you show what happens without memory, and visualisation helps...a lot.

His key implementation:
Associative Memory using Knowledge Graphs↴

Rather than storing memories in isolation, Arturo's system creates dynamic relationships between memory nodes. When the agent recalls one memory, it can traverse connected memories through semantic relationships.

Some of you might be surprised that MongoDB(even though you shouldn't be 😉) can be used as a graph database that enables graph traversal.
But it can! Arturo is leveraging MongoDB's flexible document model and aggregation pipeline to create sophisticated graph-based memory systems.

I like Arturo memory system for obvious reasons
1 Visual memory mapping - You can actually see how the agent "thinks"
2 Dynamic relationship discovery - New connections form as memories interact
3 Context-aware retrieval - Related memories surface together naturally
4 Emergent intelligence - Complex behaviors emerge from simple memory relationships

I'm working on bringing Arturo onto the MongoDB YouTube channel to dive deeper into his agent memory journey, how he's using MongoDB for graph operations, and key learnings he can share with the community.

Day 23/100 of Agent Memory 🧠

The AI Memory Meetup in SF Has Me Thinking
Maybe London Needs One Too 🤔

Just watched some incredible presentations from the AI Memory Meetup in San Francisco (June 18th, 2025), and honestly - maybe we should have one of these in London?

🧠 William F. **Founding Engineer** LangChain):
His presentation is honestly a must-watch primer on AI/Agent memory from the folks who created LangMem, LangGraph, and LangChain.
Key takeaways:
1 There's no one-size-fits-all memory solution
2 Memory is software, not hardware. In the world of Agent Memory, the database disappears; and software engineering prevails. Of course when you go a bit deeper into memory engineering and making agent memory performant, then key database concerns emerges again. But we are entering a paradigm shift, that although not new, is creating new ways to approach data management in AI Agents.
3 It's part of a broader context system - Memory doesn't exist in isolation. I have been debating in my head on my commutes into the office, does context engineering encapsulate memory engineering or is it the over way round. I will spend this weekend fleshing out my thoughts, but I need to find my answer to this soon.

💻 Yash Gaitonde (**Memory Engineer @ Cursor**):
This talk was impressive on multiple levels.
Most striking?
He's the only one working on Cursor's memory feature at a $9B company where memory is load-bearing infrastructure.
Some takeaways
→ Task-specific vs generalizable memory distinction
→ "50% junk memories are fine" - models filter noise effectively. I see a lot of memory management approaches using LLMs for summarization and observations to extract key aspects of extensive content within context window.

I actually like the reference as his title as: Memory Engineer in the video descriptions. I want to see more memory engineers!!!!!

My reflection:
I've been a massive advocate for Agent Memory, and of course whoever solves memory in AI agents is winning a massive portion of AI mindshare.

But honestly?
I don't actually know what "solving memory" looks like.
Not yet anyways

What I do know: Everyone in the AI stack is working on memory in some form. It's the one problem on the path to AGI that everyone can participate in.
We can't all build foundational models (not with my Mac laptop anyway).
But memory engineering?
That's democratized.

Agent Memory might just be the great equalizer in the race to AGI for some players in the AI Stack.
https://lnkd.in/escejZkV
https://lnkd.in/eY9KgGPc

Day 24/100 of Agent Memory 🧠

Everyone seems to be doing this in Agent Memory↴
One method that I've seen pretty much everyone in the space of memory engineering and those tackling agent memory is summarization.

It's become the de facto approach for managing memory overflow and knowledge compression.

There is no one way to do summarization, but with the improvement in model reasoning abilities and reduction of LLM inference latency, I think a lot of approaches to summarisation are going to be utilizing LLMs and how good one is over the other will be based on prompts provided(check out mine in the attached image).

Common Approaches:

🔲 Context Window Overflow Summarization ↴
Most systems trigger summarization whenever the context window is about to overflow - when content is about to exceed token limits. It's reactive compression.

⏰ End-of-Session Summarization ↴
Some systems perform summarization at the end of a session or "day." This approach was prominently showcased in Stanford's Generative Agents paper, where agents periodically reflected on their daily experiences and distilled them into higher-level insights. These reflections helped shape their memory, influence future decisions, and maintain coherent long-term behavior.

🛠️ Agent-Controlled Summarization ↴
Some libraries give summarization as a tool to the agent, allowing the LLM to decide when to summarize based on contextual relevance rather than arbitrary limits.

In MemoRizz, I've implemented a unique approach:
My _compress_memories_with_llm() method focuses on emotionally and situationally relevant compression:

→ Extracts situationally relevant context and patterns
→ Maps relationship dynamics and behavioral patterns
→ Captures emotionally significant moments and interactions
→ Identifies key achievements, challenges, and learning experiences

But here's where it gets interesting:
In MemoRizz, I use these summaries to dynamically update the agent's persona. The _update_persona_with_llm() method analyzes recent experience summaries and evolves the agent's goals and background based on:

→ Refined understanding of capabilities
→ New skills or knowledge gained
→ Evolved priorities or interests
→ Adjusted behavioral patterns

The key insight: I use summarisation memory units to compress experiences to fundamentally change how the agent sees itself and its capabilities. Summarisation

memory units in the MemoRizz library are time stamped, so these fall into the episodic memory type.

Here is the file location of the code for summarisation in MemoRizz: https://lnkd.in/eveBxtRw

```
    def _update_persona_with_llm(self, summaries: List[Dict]) -> Dict[str, str]:
        """
        Use LLM to update persona based on summaries.

        Parameters:
        -----------
        summaries : List[Dict]
            List of summary documents

        Returns:
        --------
        Dict[str, str]
            Dictionary containing updated 'goals' and 'background' fields
        """
        try:
            # Extract summary contents
            summary_contents = [s.get('summary_content', '') for s in summaries if
s.get('summary_content')]

            if not summary_contents:
                return {}

            summaries_text = "\n\n".join(summary_contents)

            # Current persona information
            current_goals = self.persona.goals
            current_background = self.persona.background

            # Create persona update prompt
            update_prompt = f"""
            Based on the following recent summaries of experiences and interactions,
            analyze how the persona should evolve and update the goals and background accordingly.

            Current Persona:
            Goals: {current_goals}
            Background: {current_background}

            Recent Experience Summaries:
            {summaries_text}

            Please provide updated persona information that reflects growth, learning, and adaptation
based on these experiences.
            Consider:
            - New skills or knowledge gained
            - Evolved priorities or interests
            - Adjusted behavioral patterns
            - Refined understanding of capabilities

            Respond with ONLY a JSON object in this exact format:
            {{
                "goals": "updated goals here",
                "background": "updated background here"
            }}
            """

            # Use the agent's LLM to generate updates
            response = self.model.generate_text(
                update_prompt,
                instructions="Return only a valid JSON object with 'goals' and 'background' fields. No
other text."
            )

            # Parse JSON response
            import json
            try:
                persona_updates = json.loads(response.strip())
                if isinstance(persona_updates, dict) and 'goals' in persona_updates and 'background' in
```

```
            import json
            try:
                persona_updates = json.loads(response.strip())
                if isinstance(persona_updates, dict) and 'goals' in persona_updates and 'background' in
persona_updates:
                    return persona_updates
                else:
                    logger.warning("LLM response was not in expected format")
                    return {}
            except json.JSONDecodeError:
                logger.warning("LLM response was not valid JSON")
                return {}

        except Exception as e:
            logger.error(f"Error updating persona with LLM: {e}")
            return {}
```

Day 25/100 of Agent Memory 🧵

Spending this weekend doing some more reading on Agent Memory. Going through my thought process on agent memory. I've refined my initial definitions of agent memory ↴

Agent Memory (and Memory Management) is a computational exocortex for AI agents—a dynamic, systematic process that integrates an agent's LLM memory (context window and parametric weights) with a persistent memory management system to encode, store, retrieve, and synthesize experiences. Inspired by human cognitive memory, it enables agents to accumulate knowledge, maintain conversational and task continuity, and adapt behavior based on history—making them more reliable, believable, and capable.

💡What do I mean by Exocortex?
Exocortex = an external cognitive layer that augments an agent's native "mind."
For AI, a computational exocortex is everything outside the LLM that extends its memory and reasoning

💡Why is it "systematic"?
Because it's not just "remembering things"; it's a repeatable, governed process with clear stages, rules, best practices and metrics.
Day 26/100 of Agent Memory 🧵

I really need to get better at recording courses.
Until then, I do my best work live.

So that's why
This Wednesday ↴
4-Hour Deep Dive into Agent Memory (100+ Slides Ready!)"

This Wednesday:
4-Hour Deep Dive into Agent Memory (100+ Slides Ready!)"

After 25 days of exploring Agent Memory together, it's time to take this conversation to the next level. This Wednesday, I'm doing a 4-hour live online training session with O'Reilly on AI Memory Management in Agentic Systems.

📎 Join me: https://lnkd.in/eKinRhFE

What I've prepared:
I just submitted a presentation deck with 100+ slides covering everything we've discussed and much more. We're going deep into the architecture, implementation, and real-world challenges of memory engineering.

My 2025 mission: Create 100 memory engineers (still trying to figure out how I'll track this lol... but hey, we'll get there!). Looks like we have around 85 memory engineers that have signed up...does this count if everyone shows up....let's decide on this later.

Back to the Live training...

The journey I'll take you on:
From: "Hey, what is this thing that generates text when I prompt it?"
To: "Hey, this thing can remember conversations we had last week, has the personality of my favorite actor, and seems very reliable, believable, and capable."

What we'll cover:
→ Key concepts: Agent Memory, Memory Engineering, cognitive architectures
→ Retrieval mechanisms: Common patterns in RAG and Agents beyond vector similarity
→ Memory management solutions: Deep dives into real implementations
→ Library exploration: LangMem from LangChain, MemoryBank from Google, Session Management from OpenAI SDK and more.
→ Bonus: Exploration of the MemoRizz codebase and design patterns for common memory types.

My aim for this session: Equip you with the knowledge, tools, and confidence to architect and implement production-ready memory systems that transform reactive AI applications into truly intelligent agents.

Day 27/100 of Agent Memory 🧠

That Was Absolutely Insane
4 Hours Live Training session on Agent Memory

That Was Absolutely Insane
4 Hours Live Training session on Agent Memory
Thanks for the support O'Reilly

I just finished probably one of the longest teaching session I've ever done.
I haven't slept in 24 hours, and I've been speaking non-stop for 5 hours.
But man I am soo energized.

What is actually insane is that over 1,000 learners registered for the session.

It was one of those "you had to be there" moments (not really, because O'Reilly will release the recording soon).

Here's what I touched on:
☆ Module 1: Explained key terms and their definitions - AI Agents, Agent Memory, Agent Memory Management, Memory Engineering and more. This part was about creating common ground for everyone that we'd build all our technical knowledge on. We explored various memory types, their definitions and applications.

☆ Module 2: Information retrieval is all you need. Explained all the retrieval techniques memory engineers need to consider - lexical search, vector search, hybrid search, plus the indexes associated with these mechanisms. We explored RAG and Agentic RAG, then went through the zero-to-hero notebook taking you from information retrieval mechanisms to RAG pipelines to Agents and Multi-Agent systems.
📎 Link: https://lnkd.in/et42sGu8

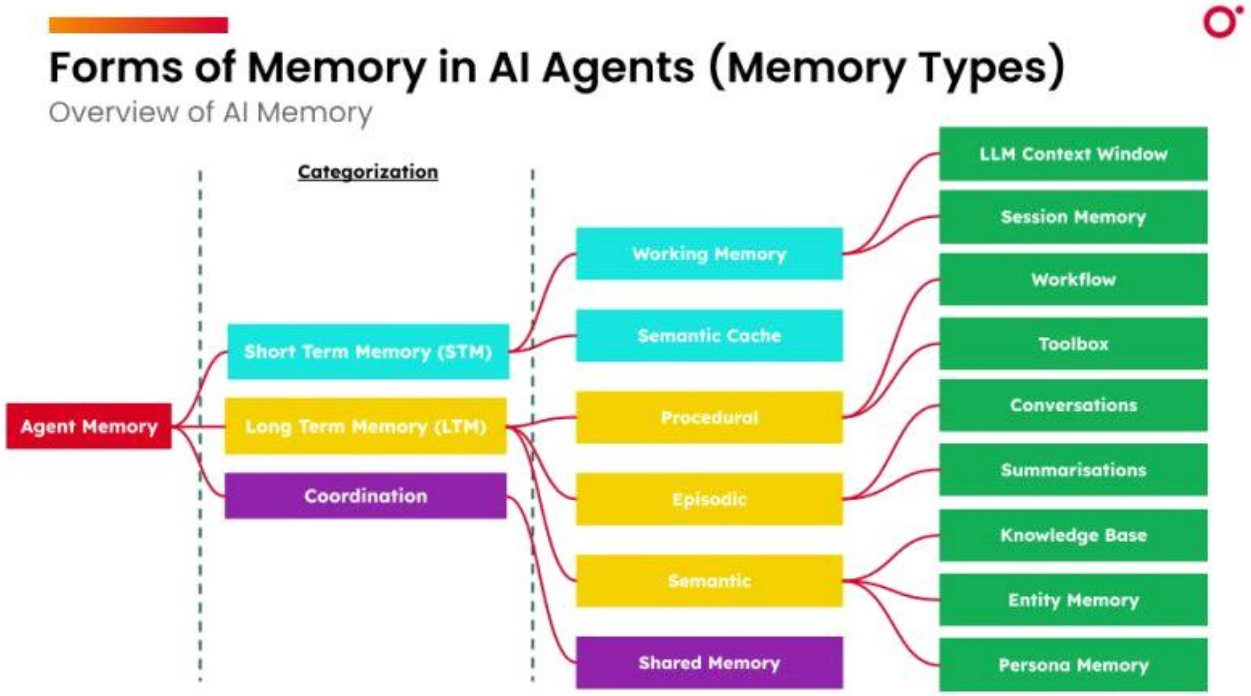
☆Module 3: This is where the fun starts (kidding, it already started). We dove deep into memory management solutions - leveraging LangChain's LangMem and LangGraph for building memory-augmented agents, followed by step-by-step code using Mem0 and Google's MemoryBank. All with MongoDB as the Memory Provider for AI Agents.

☆ Module 4: Deep Dive into MemoRizz
This was the crown jewel - walking through the complete MemoRizz architecture live. We explored the cognitively-inspired memory system (semantic, procedural, episodic), demonstrated real-time persona evolution through summarization, showed multi-agent coordination patterns, and went through production-ready code that you can actually use today. Watching people's "aha moments" as they saw how all the concepts clicked together in a working system was absolutely magical.

The energy in the session was impressive, and glad folks stayed for the full 4 hours.
Give me a shout in the comments if you were there 😉

The energy in the session was impressive, and glad folks stayed for the full 4 hours.
Give me a shout in the comments if you were there 😉

Resources:
📎 Code: https://lnkd.in/eFybeXju (hey, the stars are going ☝)
📎 Presentation: https://lnkd.in/e_jz6PXv
📎 Discord Server For Memory Engineers: https://lnkd.in/eQ7z7BWy



Day 28/100 of Agent Memory 🥹

People always ask me
What Library should I use to handle memory in AI Agents?
I don't have a preference, but in close first is⤵

LangMem, LangGraph, and LangChain
The LangChain team certainly understand the space of AI application development on various level.

Just finished an exploration of LangMem, LangGraph, and LangChain for building memory-augmented agents. The combination is intuitive, and I've got some thoughts.

🛠 **How Each Library Handles Memory**:
→ LangMem: Memory management tools (create_manage_memory_tool, create_search_memory_tool) that give agents the ability to actively store and retrieve memories across different namespaces.

→LangGraph: Agent orchestration with built-in checkpointing via MongoDBSaver for conversation state persistence and create_react_agent for memory-augmented workflows.

→LangGraph: Agent orchestration with built-in checkpointing via MongoDBSaver for conversation state persistence and create_react_agent for memory-augmented workflows.

→ LangChain: The foundation layer providing embeddings, document processing, and integration capabilities that power the entire memory stack.

🧠 **Memory Type Mapping**:
→ Procedural Memory: Agent instructions and tool configurations stored in MongoDB collections
→ Semantic Memory: Knowledge base chunks with vector search via VoyageAI embeddings
→ Episodic Memory: Conversation history managed through LangGraph checkpointing
→ Working Memory: Dynamic context assembly through intelligent prompt engineering

💡 **Key Learnings**:
1 There are two Retrieval Approaches:
Eager retrieval (always inject relevant knowledge) vs tool-based retrieval (let agent decide when to search).
Both have trade-offs in latency vs precision.

2 Memory Namespace Design: Organizing memories by purpose (user_memories, agent_memory_survey, conversations) creates clean separation and better retrieval accuracy.

3 Tool Registry Scaling: Dynamic tool creation beats static arrays when dealing with multiple memory types. The semantic tool retrieval mechanism is also intuitive and one that I always advocate for my customers to implement.

What really impressed me: The incorporation of a mechanism to use existing memory units to update the instruction/procedural memory of the agent.

```
optimizer_result = optimizer.invoke({
 "prompt": current_prompt,
 "trajectories": [(response["messages"], feedback)]
})
```

This is continuous learning and adaptation in action. The agent uses memories to evolve its own operating instructions.
That's memory engineering.

My opinion:

My opinion:
This stack is feature complete but requires thoughtful architecture.
The MongoDB integration is solid, the memory tools are intuitive, but you need to carefully design your namespace strategy and retrieval patterns.
It's not plug-and-play.

📎 Full exploration: https://lnkd.in/eR5TNpvU

```python
from langchain_voyageai import VoyageAIEmbeddings
from langgraph.store.mongodb.base import MongoDBStore, VectorIndexConfig
from pymongo import MongoClient
from langmem import create_manage_memory_tool, create_search_memory_tool
from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.mongodb import MongoDBSaver

client = MongoClient(MONGODB_URI)
db = client[DATABASE_NAME]
procedural_collection = db[PROCEDURAL_MEMORY_COLLECTION_NAME]
semantic_collection = db[SEMANTIC_MEMORY_COLLECTION_NAME]

procedural_vector_index_config = VectorIndexConfig(
    dims=1024,
    index_name="procedural_memory_index",
    filters=None,
    fields=None,
    embed=VoyageAIEmbeddings(model="voyage-3-large"),
)

semantic_vector_index_config = VectorIndexConfig(
    dims=1024,
    index_name="semantic_memory_index",
    filters=None,
    fields=None,
    embed=VoyageAIEmbeddings(model="voyage-3-large"),
)

procedural_memory_store = MongoDBStore(
    collection=procedural_collection,
    index_config=procedural_vector_index_config,
    auto_index_timeout=70
)
semantic_memory_store = MongoDBStore(
    collection=semantic_collection,
    index_config=semantic_vector_index_config,
    auto_index_timeout=70,
)

checkpointer = MongoDBSaver(
    client=client,
    db_name=DATABASE_NAME,
    collection_name=STATE_CHECKPOINT_COLLECTION_NAME,
    index_config=procedural_memory_store
)

def simple_prompt(state):
    """Simple prompt that only includes procedural instructions"""
    item = procedural_memory_store.get((("instructions",), key="agent_instructions")
    instructions = item.value["prompt"]
    sys_prompt = {"role": "system", "content": f"## Instructions\n\n{instructions}\n\nYou have access
to search tools for knowledge retrieval when needed."}
    return [sys_prompt] + state['messages']

memory_tools = [
    create_manage_memory_tool(USER_MEMORY_NAMESPACE),    # User-specific memories
    create_search_memory_tool(USER_MEMORY_NAMESPACE),    # Search user memories
    create_search_memory_tool(KNOWLEDGE_NAMESPACE),      # Search PDF knowledge base
]

response = chat_with_agent(
    agent=agent,
```

```
]

response = chat_with_agent(
    agent=agent,
    query="Who are the authors of the paper A survey on the memory mechanism of large language
models?",
    thread_id="10",
    user_id="user-123",
    return_full_result=True
)
```

Day 29/100 of Agent Memory 🧠

Just finished exploring Mem0
This library aims to be the memory layer for AI Agents.
And they seem to be on a good path to achieving that goal.

The library is an easy entry for Memory Engineers who want to get a PoC built quickly and don't want to worry about the details of implementing custom memory types, and memory operation logic.

You can get started with the library in about 5 lines of code and you will get:
✅ Dual memory stores: Extracted memories (user preferences) + Semantic memory (knowledge base)
✅ User-specific memory: Isolated memory spaces per user
✅ MongoDB integration: Vector search with automatic indexing
✅ Simple API: memory.add(), memory.search() - clean and intuitive

🚀 **Advanced Memory Operations**:
The library also offers some impressive advanced features:
→ Complex filtering: AND, OR, NOT logical operators with custom search filters → Dual agent/user memories: Separate memory spaces for users and agents in same conversation
→ Memory history tracking: See how memories evolve and change over time
→ Advanced search: Filter by metadata, categories, date ranges, and custom criteria
→ Natural language delete: "Delete all my food preferences" - I like this one...but I wonder what the failure rate of this is 🤔.
→ Entity management: Track all users, agents, and runs with associated memories.
Entity memory is something I need to implement in MemoRizz, but again, pretty cool to see all the logic for complex memory management built out of the box

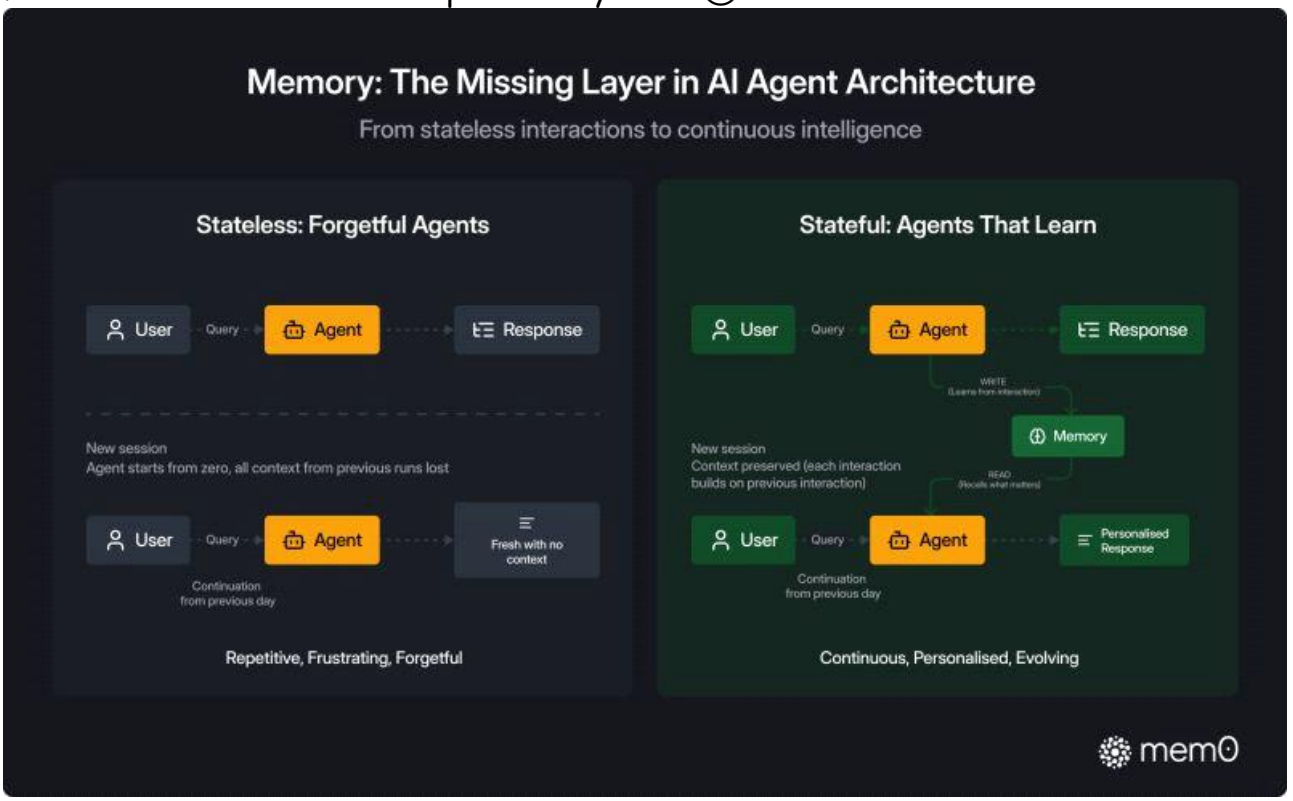
In my example (code in comments) I do the following
🧠 **Memory Type Mapping**:
→ Extracted Memories: "Likes to drink tea", "Particularly likes green tea" - user preferences
→ Semantic Memory: Research paper chunks with vector search capabilities

→ Extracted Memories: "Likes to drink tea", "Particularly likes green tea" - user preferences
→ Semantic Memory: Research paper chunks with vector search capabilities
→ Conversation Memory: Automatic storage of chat interactions with context

🗄 **Key Learnings**:
1 Dual Memory Strategy Works: Separating user preferences from knowledge base creates cleaner retrieval patterns. When I asked about tea, it found my preferences. When I asked about the paper, it found relevant chunks.
2 Automatic Memory Extraction: Mem0's ability to automatically extract "Likes to drink tea" from conversation is impressive. No manual memory management needed.
3 Integrations: The library integrates with widely known databases, inlcuding MongoDB

A question you are probably thinking is ↴
Mem0 vs LangMem
Although I don't have a preferecne, I would say it's a decision between: **Automation vs Control**
LangMem gives you control
Agents explicitly decide when to store/retrieve memories through tools.
Mem0 prioritizes automation
It extracts preferences from conversations without explicit commands.

For now I would advise developers to try both 😉



Day 30/100 of Agent Memory 🧠
Explored Google's ADK memory management implementation and discovered they

Day 30/100 of Agent Memory 🧠

Explored Google's ADK memory management implementation and discovered they offer two distinct approaches: InMemoryMemoryService and VertexAiMemoryBankService. I focused on the InMemoryService since the VertexAiMemoryBankService requires full commitment to Google Cloud infrastructure.

Using memory in the Google Agent Development Kit is straight forward, especially the InMemoryService (code in the comment section)

🛠 **InMemoryService Benefits**:
→ Zero setup complexity - works out of the box for prototyping
→ Session-based memory model - stores complete conversation contexts
→ Cross-session retrieval - access memories from previous conversations
→ Enterprise architecture patterns - proper service separation and state management
→ Multi-memory support - agents can access multiple memory services simultaneously

Comparing Approaches:
LangMem gives surgical control with tool-based memory operations. Mem0 offers automatic extraction with simpler APIs. Google's InMemoryService takes the session-first approach - treating entire conversations as memory units rather than individual snippets. This preserves conversational flow better than fragment-based approaches.

🏢 **Incumbents Enter the Memory Space**:
Memory in AI Agents is a low hanging fruits for anyone that plays in the AI Stack, and hyperscalers aren't just sitting by watching everyone else try to solve it. Google has made a solid open source approach with memory in ADK, and AWS released Agentcore, which has memory implementation, but it's within a blackbox (maybe they release a library soon 😉). Google's approach signals how tech giants plan to dominate: by offering fully managed, deeply integrated memory solutions that are technically superior but ecosystem-dependent.

🚀 **VertexAiMemoryBankService Advantages**:
For those who are within the Google ecosystem, MemoryBank offers LLM-powered memory extraction that intelligently processes conversations and consolidates memories automatically. It provides persistent storage, advanced semantic search, and memory consolidation that goes far beyond simple storage. New memories get intelligently merged with existing ones, creating evolving knowledge bases rather than static archives.

My take: InMemoryService gives developers simplicity, while

than static archives.

My take: InMemoryService gives developers simplicity, while VertexAiMemoryBankService showcases what's possible with full ecosystem adoption. The session-based memory model feels more natural than individual memory fragments, and it's one that OpenAI take with their session management in their Agent SDK (more on this tomorrow)

Memory engineering isn't just about startups and open-source libraries. The big players are bringing industrial-grade solutions that will be hard to compete with on technical merit.

```python
import asyncio
from google.adk.agents import LlmAgent
from google.adk.sessions import InMemorySessionService, Session
from google.adk.artifacts import InMemoryArtifactService
from google.adk.memory import InMemoryMemoryService
from google.adk.runners import Runner
from google.adk.tools import load_memory
from google.genai.types import Content, Part
from google.adk.tools.tool_context import ToolContext
from google.adk.tools import FunctionTool

APP_NAME = "memory_example_app"
USER_ID = "mem_user"
MODEL = "gemini-2.0-flash" # Use a valid model


session_service = InMemorySessionService()
memory_service = InMemoryMemoryService()
artifact_service = InMemoryArtifactService()

pdf_memory_agent = LlmAgent(
    model=MODEL,
    name="PDFMemoryAgent",
    instruction="You are an AI assistant that can query PDF content and use memory. "
                "Use the 'pdf_query_func' to search through PDF content. "
                "Use the 'load_memory' tool to recall past conversations. "
                "When a user asks about PDF content, use the pdf_query_func tool.",
    tools=[load_memory, pdf_query_tool]
)

async def run_pdf_scenario():
    """Run scenario with three specific questions about the paper."""

    # Create runner
    runner = Runner(
        agent=pdf_memory_agent,
        app_name=APP_NAME,
        session_service=session_service,
        memory_service=memory_service,
        artifact_service=artifact_service
    )

    # Create session for querying
    session_id = "pdf_query_session"
    await runner.session_service.create_session(
        app_name=APP_NAME,
        user_id=USER_ID,
        session_id=session_id
    )

    # Define three questions about the paper
    questions = [
        "What is the main contribution and purpose of this research paper?",
        "What methodology or approach does the paper use for their experiments?",
        "What are the key findings and results presented in this paper?"
```

```python
    questions = [
        "What is the main contribution and purpose of this research paper?",
        "What methodology or approach does the paper use for their experiments?",
        "What are the key findings and results presented in this paper?"
    ]

    ...

    # Add this session to memory for future recall
    completed_session = await runner.session_service.get_session(
        app_name=APP_NAME, user_id=USER_ID, session_id=session_id
    )
    await memory_service.add_session_to_memory(completed_session)
    print("\n✓ Session added to memory for future recall.")

    # Test memory recall in a new session
    print("\n--- Testing Memory Recall ---")
    memory_session_id = "memory_test_session"
    await runner.session_service.create_session(
        app_name=APP_NAME,
        user_id=USER_ID,
        session_id=memory_session_id
    )

    memory_query = Content(
        parts=[Part(text="Can you summarize what we discussed about the research paper's main
contributions?")],
        role="user"
    )

    async for event in runner.run_async(
        user_id=USER_ID,
        session_id=memory_session_id,
        new_message=memory_query
    ):
        if event.is_final_response() and event.content and event.content.parts:
            print(f"Memory Recall Response: {event.content.parts[0].text}")
```

Day 31/100 of Agent Memory 🧠

OpenAI Agent SDK: Session Memory Done Right

Just explored OpenAI's Agent SDK session memory implementation, and it's clear they've taken a very deliberate approach to memory management that focuses on flexible integration and ease of use.

🏗️ **OpenAI's Session Memory Architecture**:

→ Protocol-based design: Session follows a clear interface with get_items(), add_items(), pop_item(), and clear_session() .

→ MongoDB integration: Custom MongoDBSession class with async operations and proper error handling. As long as your custom session class adheres to the Session protocol, then you are good.

→ Conversation continuity: Complete conversation history automatically maintained across agent runs

→ Granular control: Pop specific items, clear sessions, or retrieve limited conversation history

→ Agent memory persistence: Sessions survive application restarts with proper database backing

database backing

🔗 **Key Differentiators**:
1 Automatic Context Injection: When you pass a session to Runner.run(), the entire conversation history is automatically included in the agent's context. No manual memory retrieval needed. This saves a lot of code compared to other libraries we've explored.
2 Session-First Philosophy: Unlike other libraries that treat memory as an add-on, OpenAI's SDK makes session memory a core part of the agent execution model.
3 Advanced Session Management: The pop_item() method allows for conversation corrections, and the clear_session() enables clean slate resets - sophisticated memory manipulation.

Comparing Approaches:
OpenAI's approach feels the most production-ready for conversational agents. Not because it is the robust memory implementation, but the opposite. The protocol-based design ensures consistency, the automatic context injection removes complexity, and the session management methods provide the control you need for real applications.

Other libraries focus on memory as a storage problem; OpenAI treats it as a workflow integration problem. The session becomes part of the agent's execution context, not something you manage separately.
At the same time, this can also be a disadvantage for developers needing more advanced memory features.

📎 Full exploration: https://lnkd.in/et42sGu8

```
from typing import List, Optional, Union
from datetime import datetime
import motor.motor_asyncio, os

class MongoDBSession:
    """Minimal MongoDB session for storing conversation history."""

    def __init__(self, session_id: str, uri="mongodb://localhost:27017", db_name="agent_memory",
coll="sessions"):
        self.session_id = session_id
        self.client = motor.motor_asyncio.AsyncIOMotorClient(uri)
        self.collection = self.client[db_name][coll]

    async def get_items(self, limit: Optional[int] = None) -> List[dict]:
        cursor = self.collection.find({"session_id": self.session_id}).sort("timestamp", 1)
        if limit: cursor = cursor.limit(limit)
        docs = await cursor.to_list(None)
        return [d["item"] for d in docs]

    async def add_items(self, items: List[dict]) -> None:
        docs = [{"session_id": self.session_id, "item": i, "timestamp": datetime.utcnow()} for i in
items]
        if docs: await self.collection.insert_many(docs)
```

```
        docs = [{"session_id": self.session_id, "item": i, "timestamp": datetime.utcnow()} for i in
items]
        if docs: await self.collection.insert_many(docs)

    async def pop_item(self, limit: Optional[int] = None) -> Optional[Union[dict, List[dict]]]:
        if not limit or limit <= 1:
            doc = await self.collection.find_one_and_delete({"session_id": self.session_id}, sort=
[("timestamp", -1)], projection={"item": 1})
            return doc["item"] if doc else None
        cursor = self.collection.find({"session_id": self.session_id}, projection={"item":
1}).sort("timestamp", -1).limit(limit)
        docs = await cursor.to_list(None)
        if not docs: return []
        await self.collection.delete_many({"_id": {"$in": [d["_id"] for d in docs]}})
        return [d["item"] for d in docs]

    async def clear_session(self):
        await self.collection.delete_many({"session_id": self.session_id})

    async def close(self):
        self.client.close()

# Example usage
research_agent = Agent(name="Assistant", instructions="Research and return relevant info.")
session = MongoDBSession("conversation_123", uri=os.environ.get("MONGO_URI"), db_name=DB_NAME)

result = await Runner.run(
    starting_agent=research_agent,
    input="Hi my name is Richmond, and I am a AI Engineer researching LLMs and Agent Memory",
    session=session
)

print("Assistant:", result.final_output)
```